

EXPERIMENT 1

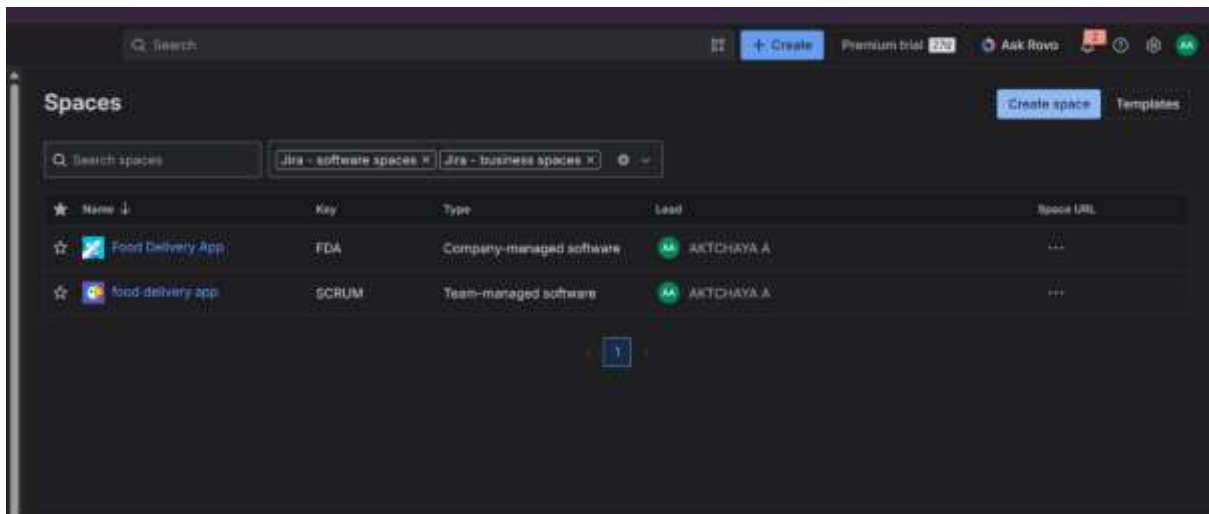
Create a Kanban Board to Visualize the Tasks.

- **Create Columns for To Do, In-Progress and Done.**
- **Add Atleast 5 Sample Tasks**
- **Move the Tasks across the Columns to Simulate the Workflow.**

Steps:

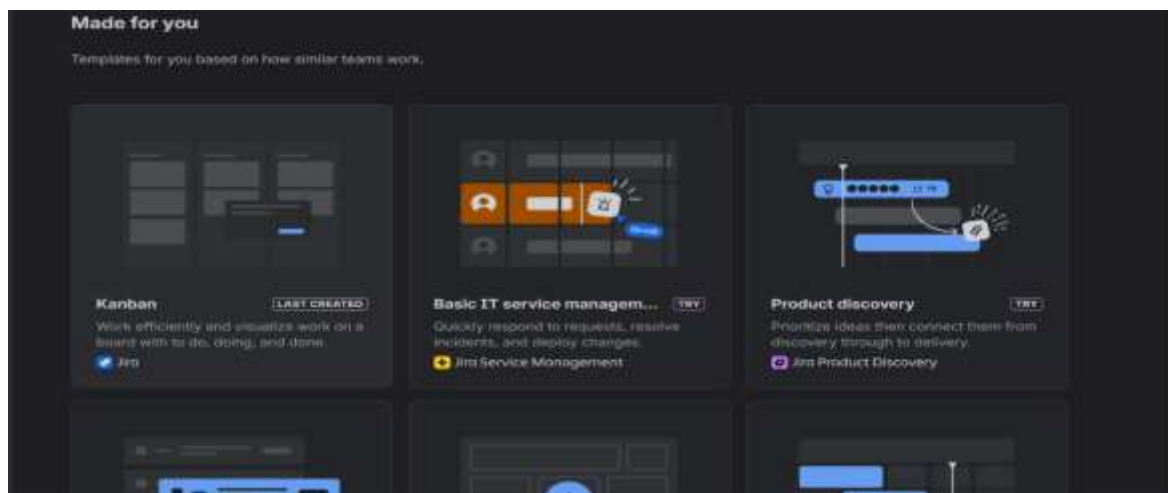
Step 1: Login to Jira Account

- Open your Google Chrome
- Visit <https://www.atlassian.com/software/jira>
- Click Log in and enter your credentials



Step 2: Create a New Project

- Click Create space
- Select Kanban
- Click Use Template
- Enter:
 - Project Name: Task Management System
 - Space to Manage: Team-Managed
- Click Create



Name your space

Required fields are marked with an asterisk *

Name *
Task Managment-1

How your space is managed *
Team-managed

Learn more about space types

Access *
Open

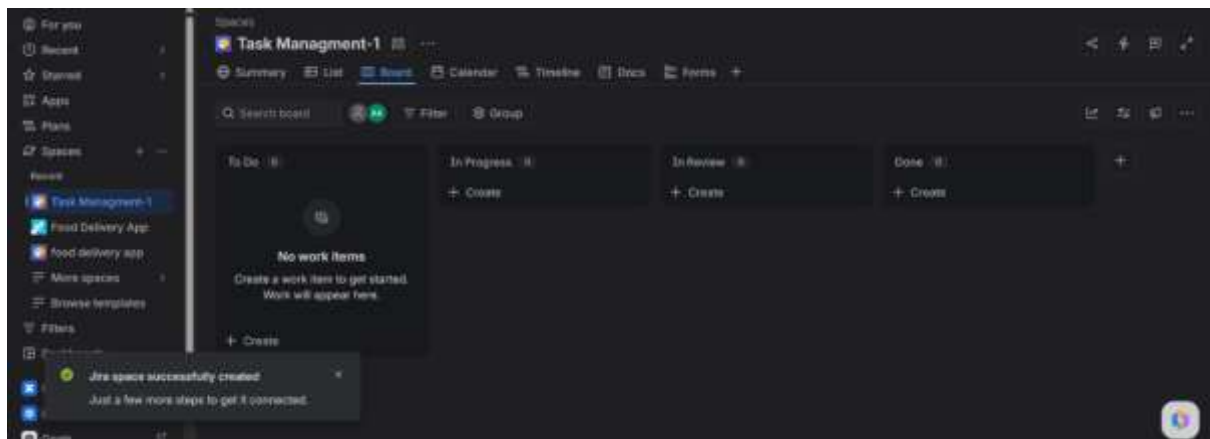
Key *
TM1

Template [See details](#)

Kanban
Jira

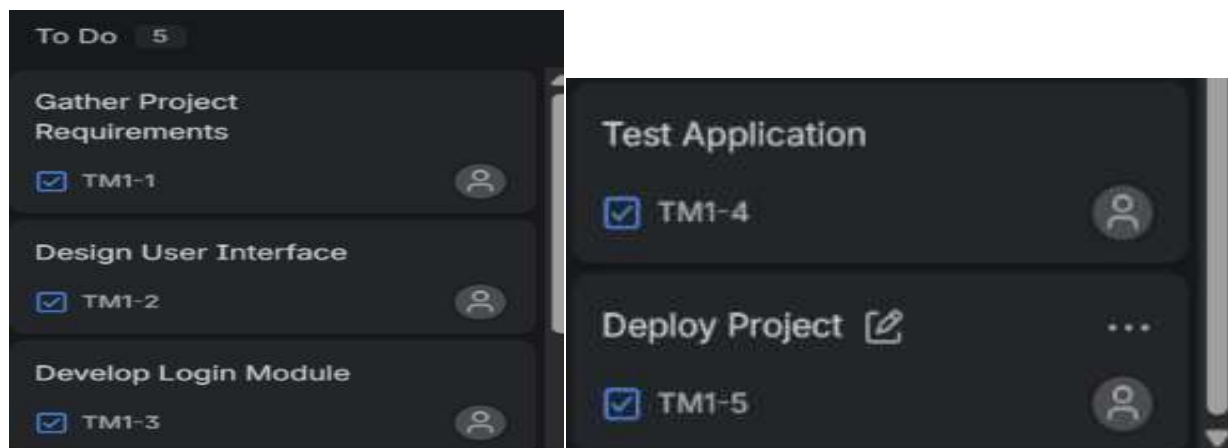
Step 3: Open the Board

- From the left menu,click Board
- The board will display three columns:
 - To Do
 - In Progress
 - Done



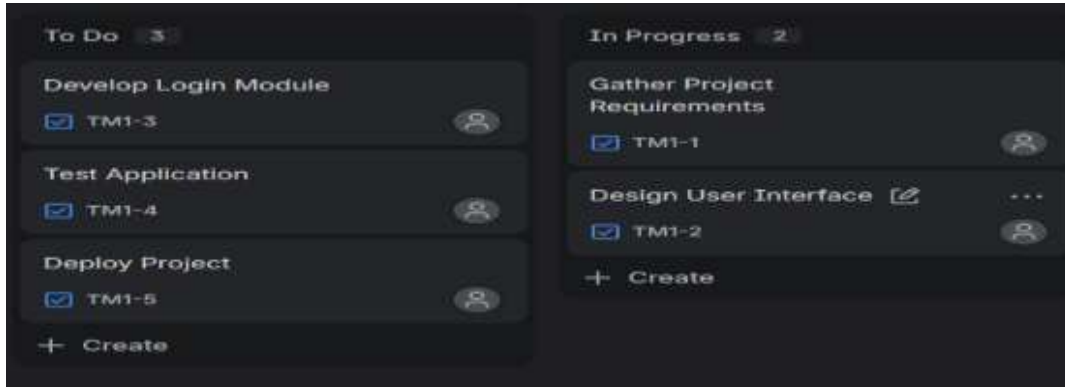
Step 4: Board with 5 Tasks

- Click Create and add any five tasks one by one
- After entering each task:
 - Click Create
 - Repeat until all five tasks are added



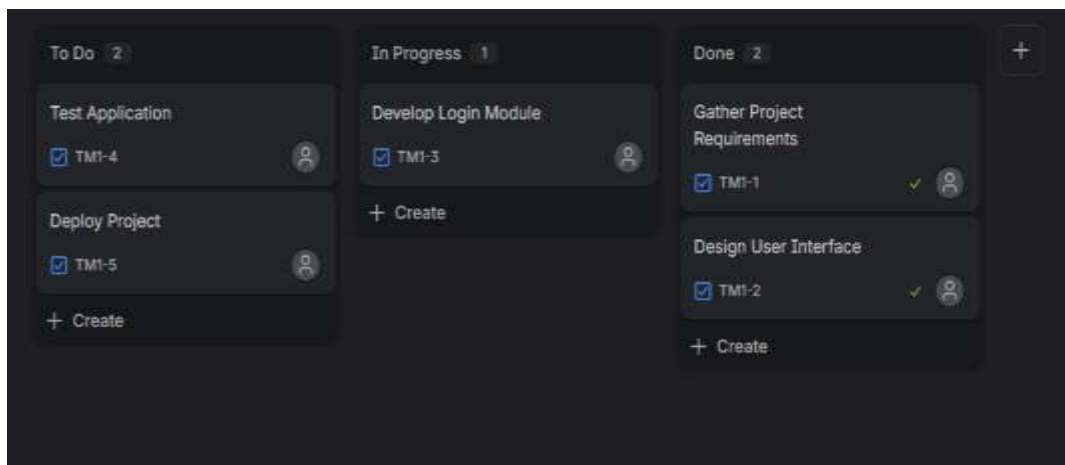
Step 5: Move Tasks to “In Progress”

- Click and hold Task 1
- Drag it to In Progress
- Repeat for Task 2
- The board should now look like:
 - To Do: Task 3, Task 4, Task 5
 - In Progress: Task 1, Task 2



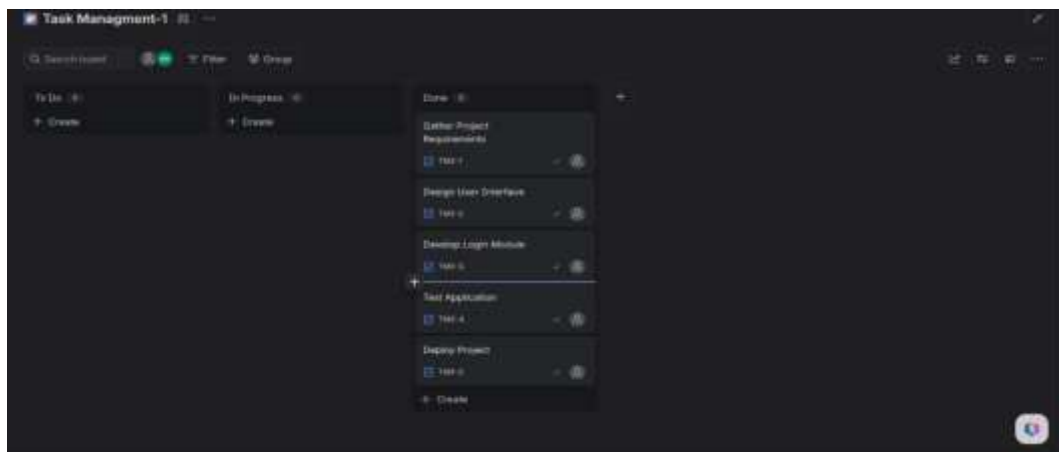
Step 6: Move Tasks to Done

- Drag Task 1, Task 2 to Done
- Drag Task 3 to In Progress
- The board should now look like:
 - To Do: Task 4, Task 5
 - In Progress: Task 3
 - Done: Task 1, Task 2



Step 7: Simulate the Workflow

Continue moving the remaining tasks one by one until all tasks reach the Done column



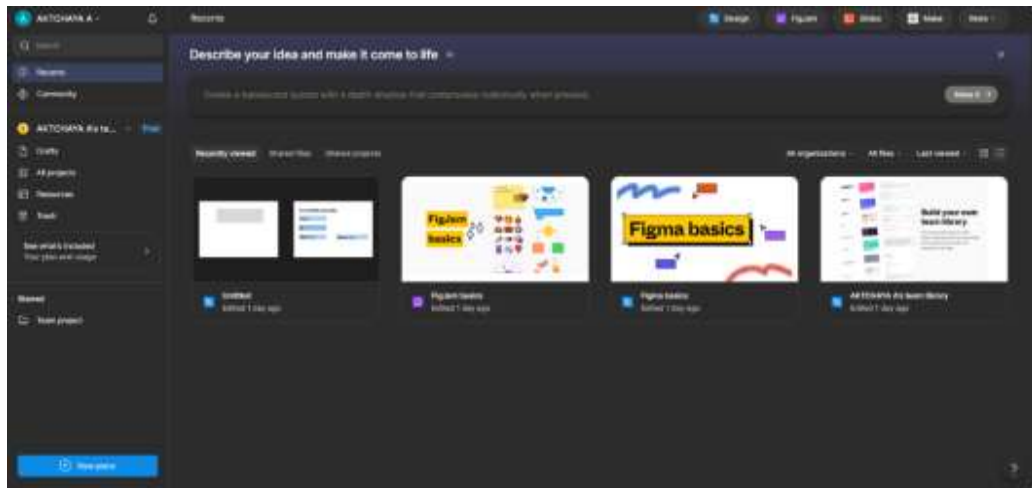
EXPERIMENT 2

Sketch a Simple Prototype of a Bus Ticket Booking System using Figma Tool

Steps:

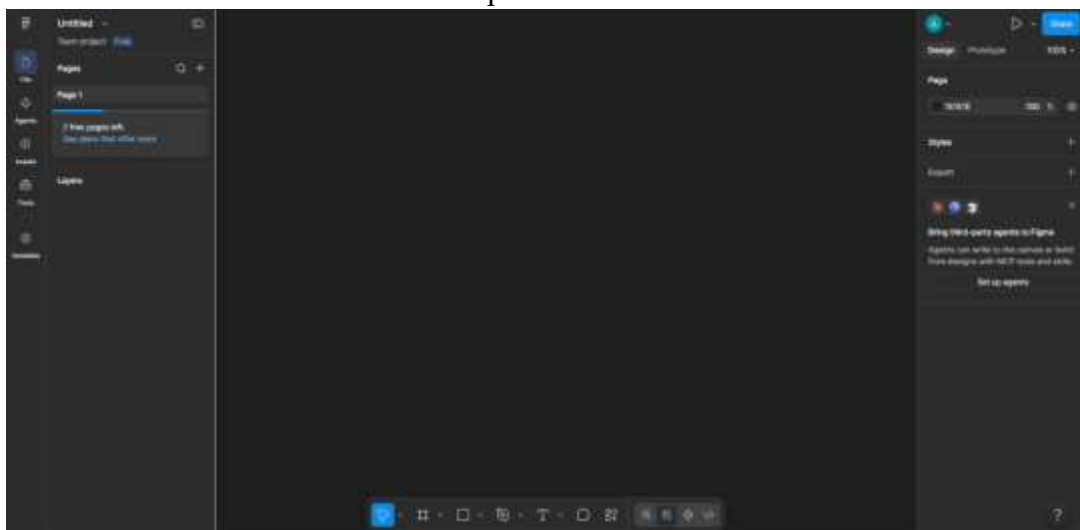
Step 1: Open Figma

- Open your Web browser
- Go to <https://www.figma.com>
- Sign in using your account



Step 2: Create a New Design

- Click New Design File
- A blank canvas will open



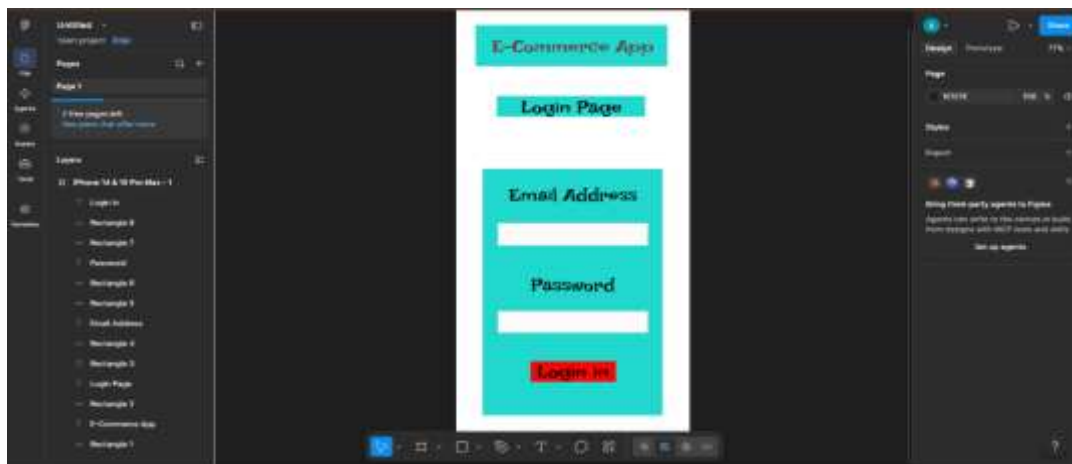
Step 3: Create a Frame

- Select the Frame Tool(F)
- Choose Phone (eg,.IPhone 14 & 15 Promax)
- A Phone frame will appear on the canvas



Step 4: Design the Login Screen

- Add a title:Bus Ticket Booking
- Insert text fields for:
 - Username
 - Password
- Add Login button



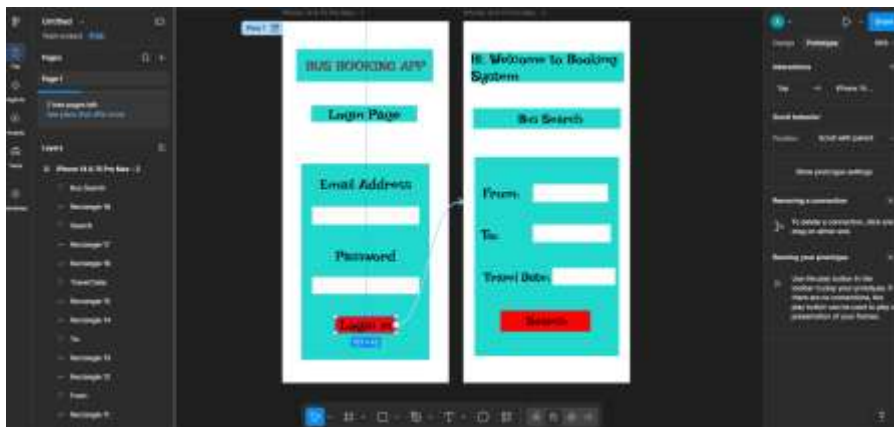
Step 5: Create the Home Screen

- Duplicate the frame
- Add:
 - From
 - To
 - Travel Date
 - Search Buses button



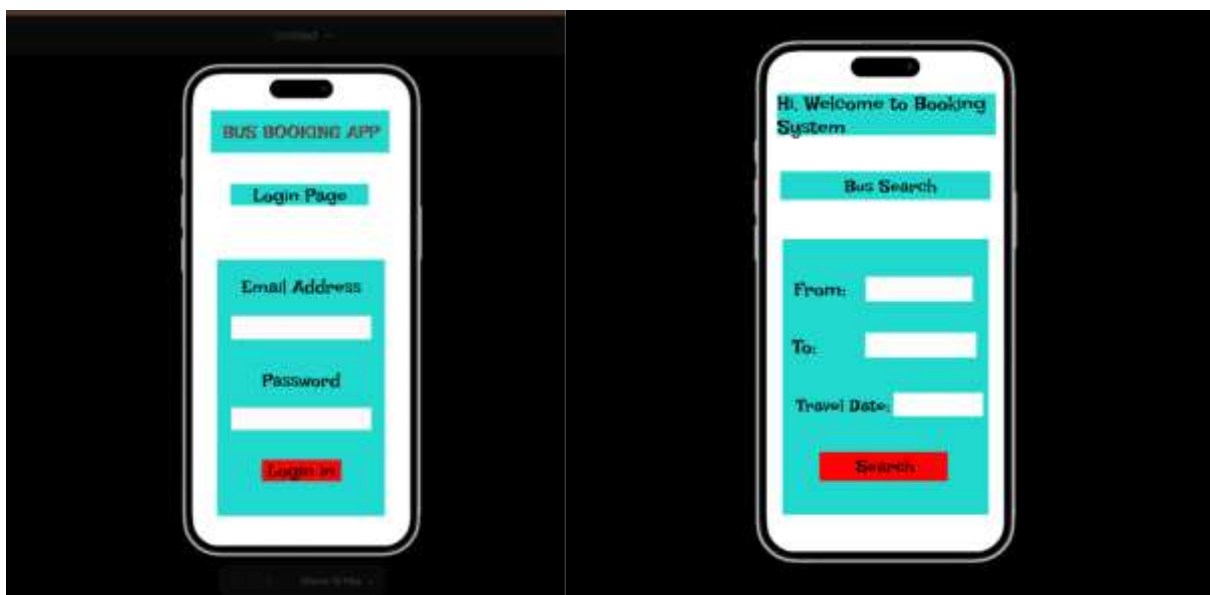
Step 6: Connect the Screens

- Switch to Prototype mode
- Connect:
 - Login → Home Screen



7. Preview the Prototype

- Click the Present (▶) button
- Test the Complete booking flow



EXPERIMENT 9

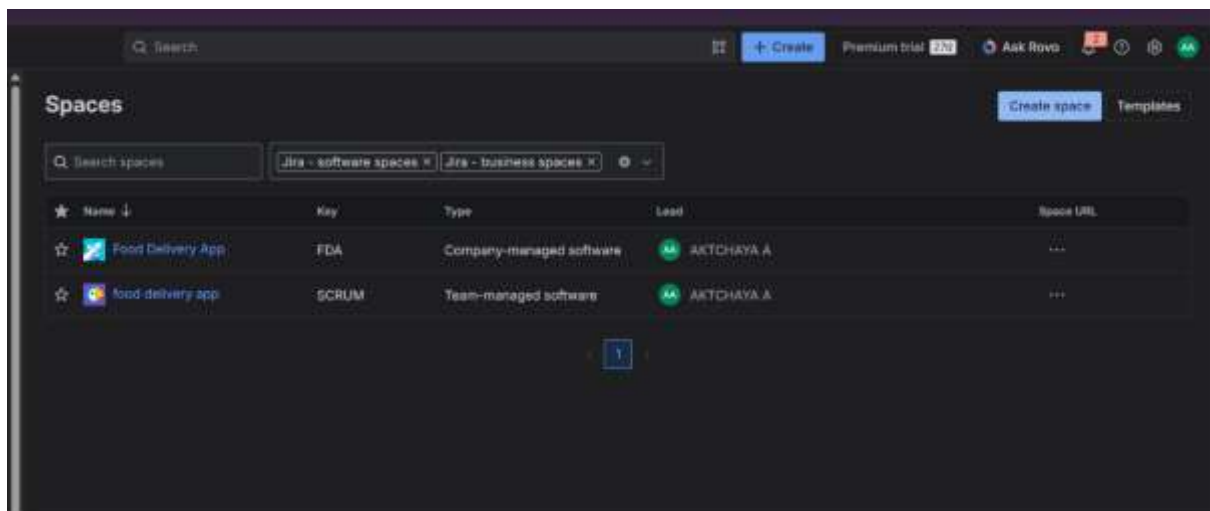
Create a Scrum Project in Jira.

- Add a backlog with at least 5 items
- Prioritize the backlog and create a 1-week sprint.
- Move backlog items into the sprint and start the sprint.
- Finally show the Screenshot of the sprint board at the start and end of the sprint

Steps:

Step 1: Login

- Open a web browser and navigate to the Jira website.
- Login - <https://id.atlassian.com/login>
- Sign in to your Jira account using your registered email address and password.
- After successful authentication, access the Jira dashboard to begin the project creation process.



Step 2: Create a Scrum Project

- Click Create Project from the Jira dashboard.
- Select the Scrum project template.
- Enter the project name as Scrum Workflow Project.
- Choose Team-managed as the project type.
- Keep the access level as Open.

- Click Create to create the Scrum project.

Name your space

Required fields are marked with an asterisk *

Name *
Task Management-1

How your space is managed *
Team-managed

Learn more about space types

Access *
Open

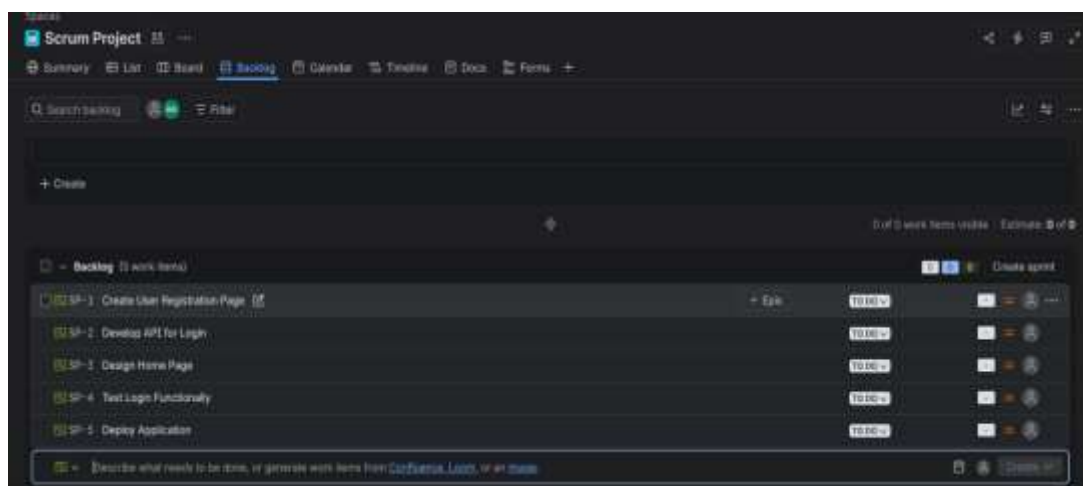
Key *
TM1

Template
Kanban

See details

Step 3: Create Backlog Items

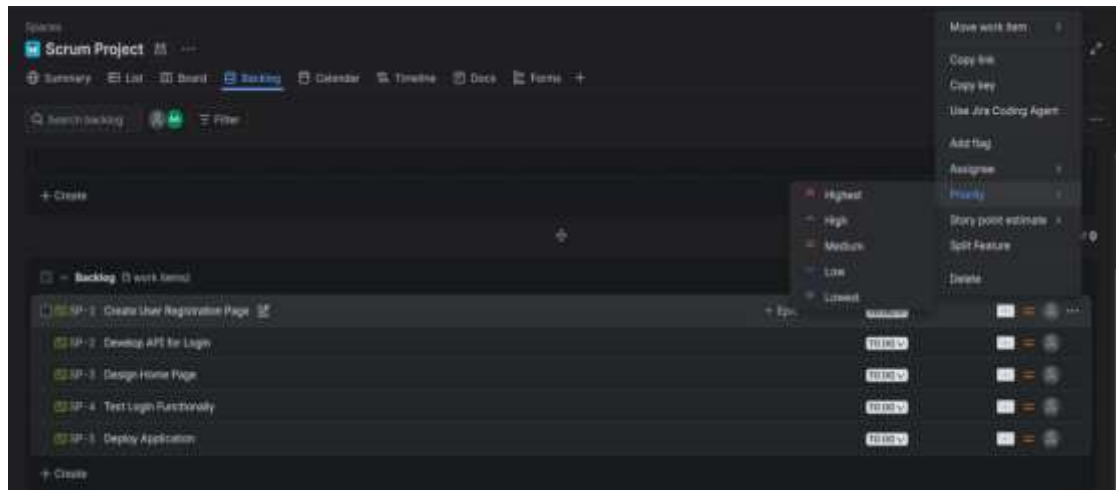
- Open the Backlog tab in the Scrum project.
- Click Create to add new backlog items.
- Create the following five backlog items:
 - Create User Registration Page
 - Develop API for Login
 - Design Home Page
 - Test Login Functionality
 - Deploy Application
- Verify that all the created items are displayed in the project backlog.



Step 4: Prioritize the Backlog

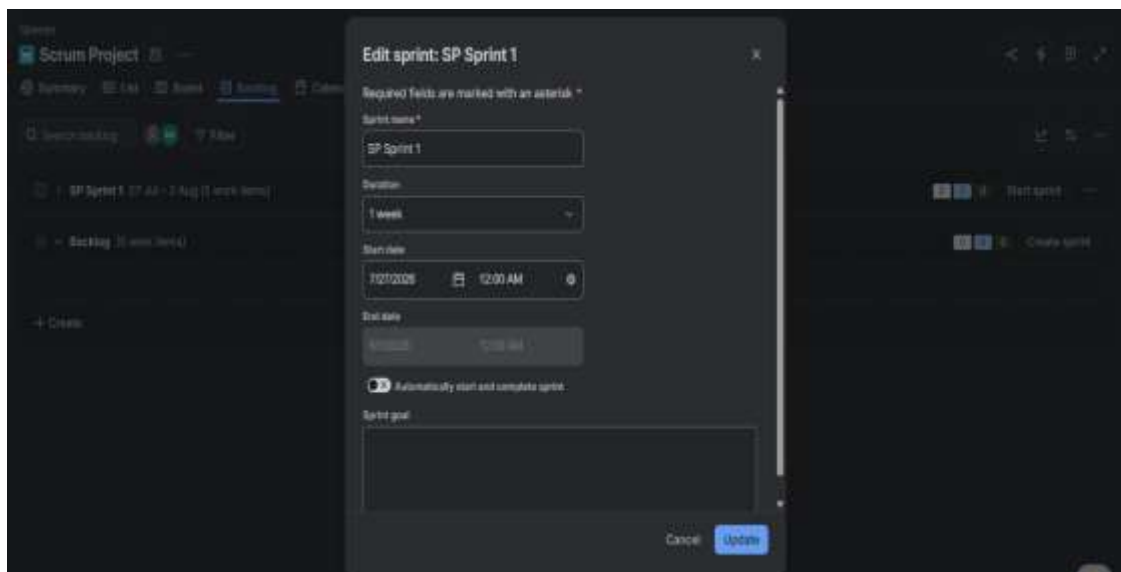
- Arrange the backlog items according to their priority by dragging and dropping them.
- Set the highest priority for Create User Registration Page and Develop API for Login.
- Place the remaining tasks in the desired order based on project requirements.

- Verify that the backlog is prioritized correctly.



Step 5: Create a 1-Week Sprint

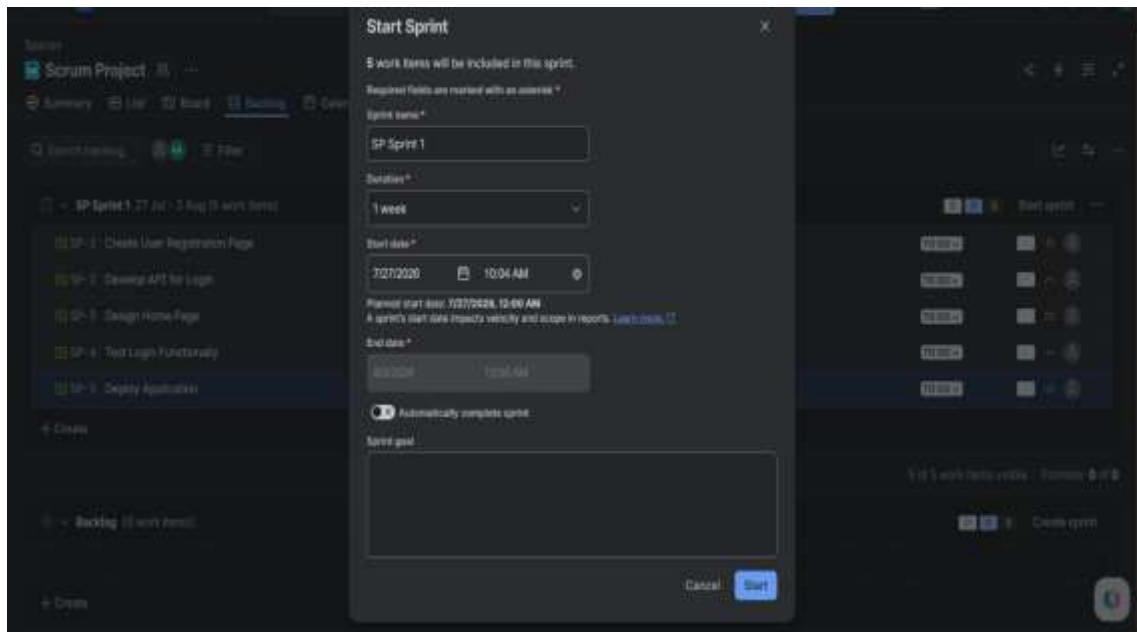
- Click Create Sprint in the Backlog view.
- Drag all five backlog items into the newly created sprint.
- Set the sprint duration to 1 week by selecting the start and end dates.
- Verify that all selected backlog items are included in the sprint.



Step 6: Start the Sprint

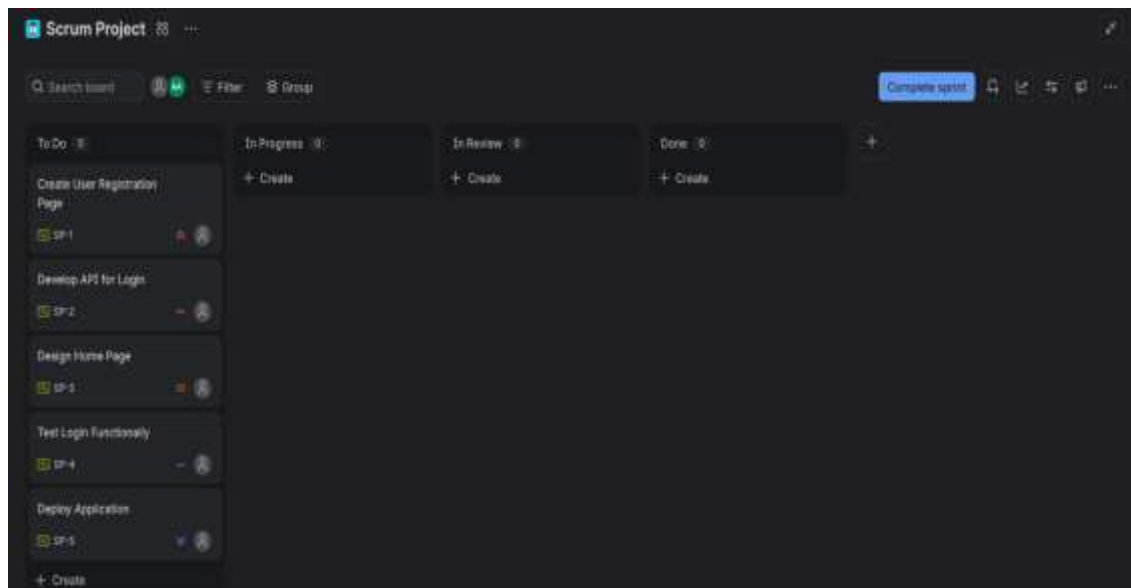
- Click the Start Sprint button.
- Confirm the sprint name and duration.
- Start the sprint to move the selected backlog items into the active sprint.

- Verify that the sprint has started successfully.



Step 7: Sprint Board at the Start

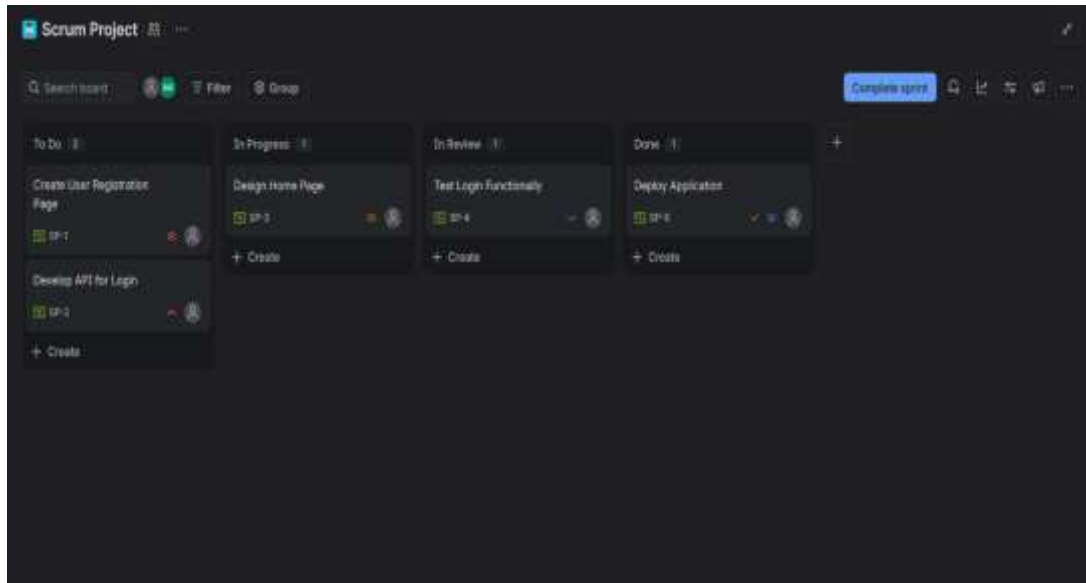
- Open the Active Sprints board.
- Verify that all sprint tasks are displayed in the To Do column.
- Capture a screenshot of the sprint board before moving any tasks.



Step 8: Sprint Board at the End

- Simulate the sprint workflow by moving the tasks across the board:
- Verify that the sprint board displays tasks in To Do, In Progress, and Done columns.

- Capture a final screenshot of the sprint board after the workflow simulation.



EXPERIMENT 11

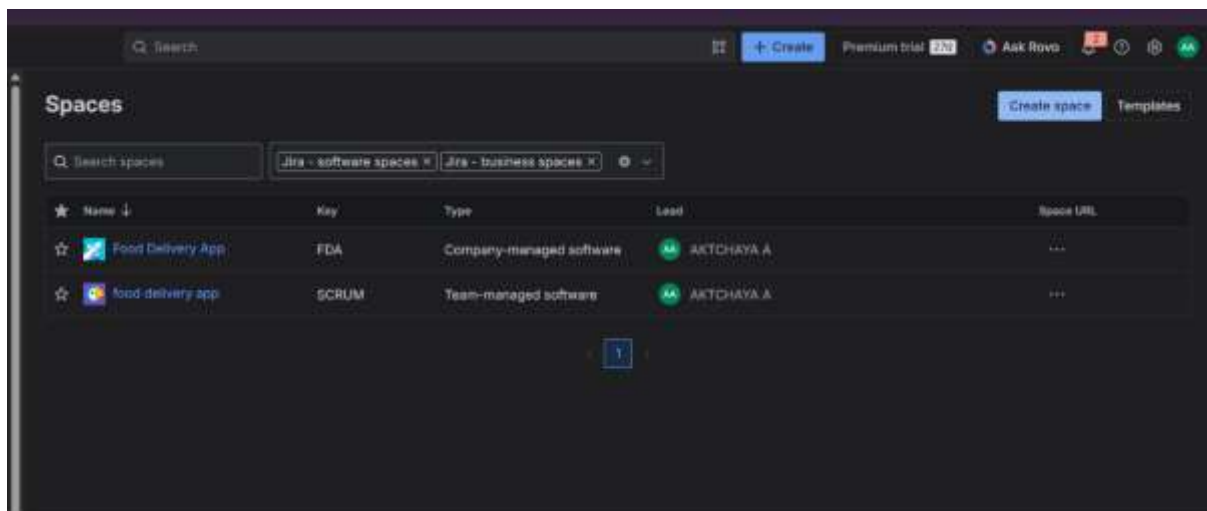
Link Jira tasks with Confluence to streamline task tracking and progress monitoring for the Library Management System development.

- **Create a new page in Confluence titled "Library Management System Project Overview**
- **Embed at least 5 Jira issues related to the development of the Library Management System**
- **Use the Jira macro to display issues with status**
- **Add a progress bar in the Confluence page to visually track the completion of each embedded Jira task**
- **Submit a screenshot of the Confluence page showing the embedded Jira tasks and the progress bar.**

Steps:

Step 1: Login to Atlassian

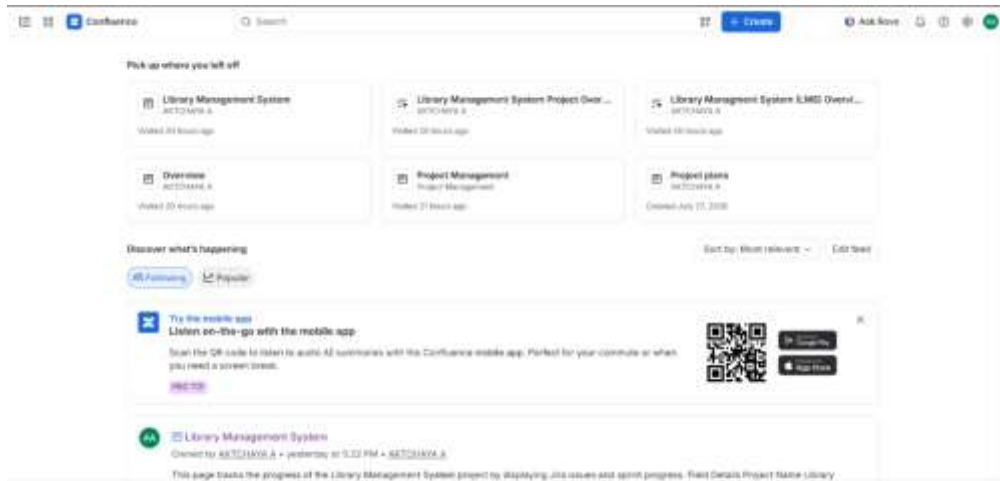
1. Open your web browser.
2. Go to the Atlassian website.
3. Sign in using your Atlassian account.
4. Open both Jira and Confluence.



Step 2: Open Confluence

1. Click Confluence from the Atlassian applications.
2. Open your desired Space.

3. Click Create to create a new page.



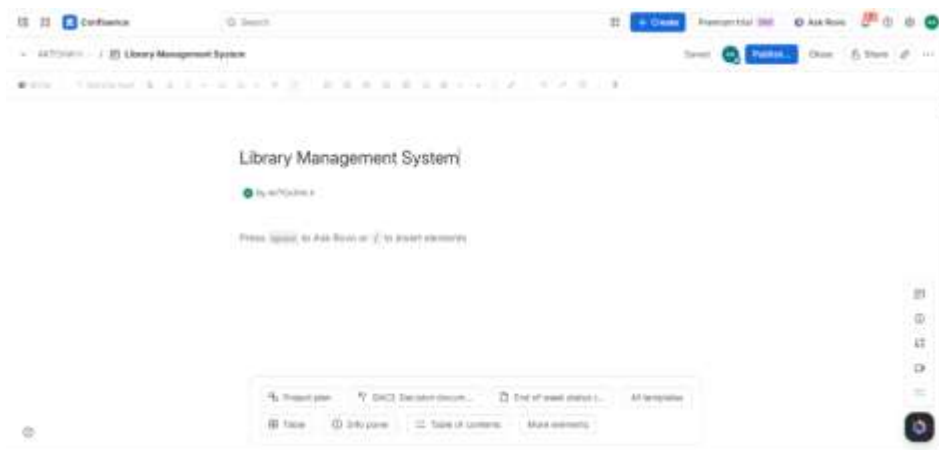
Step 3: Create a New Page

1. Select Blank Page.

2. Enter the page title:

Library Management System Project Overview

3. Leave the page body empty for now.

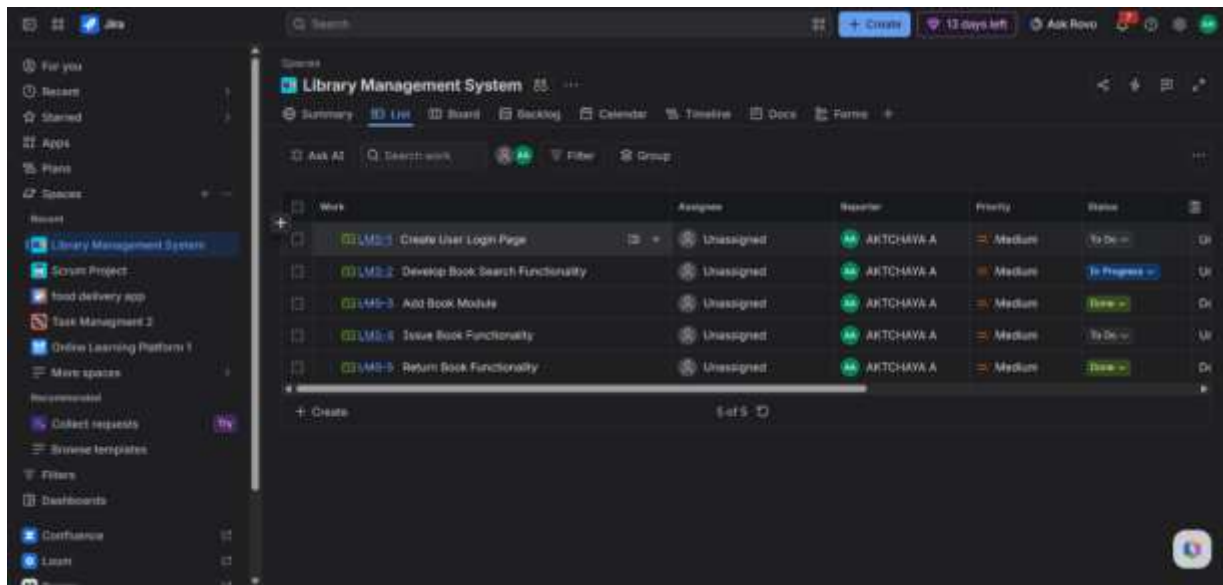


Step 4: Create the Jira Project and Work Items

- Go to Jira again
- Click Projects → Create Project
- Select Scrum Project
- Enter the project Name as Library Management System
- Click Create
- Create the following five Task in Backlog column

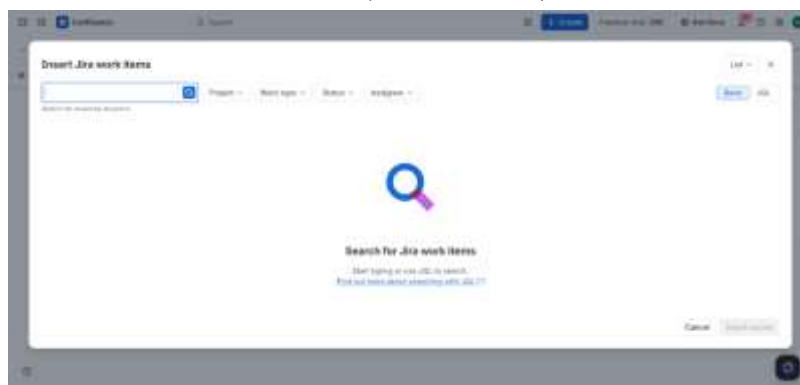
Issue Key	Summary	Status
LMS-1	Create User Login Page	To Do
LMS-2	Develop Book Search Functionality	In Progress

LMS-3	Add Book Module	Done
LMS-4	Issue Book Functionality	To Do
LMS-5	Return Book Functionality	Done



Step 4: Insert Jira Macro

1. Click inside the page.
2. Type
`/jira`
3. Select Jira Work Items (or Jira macro).



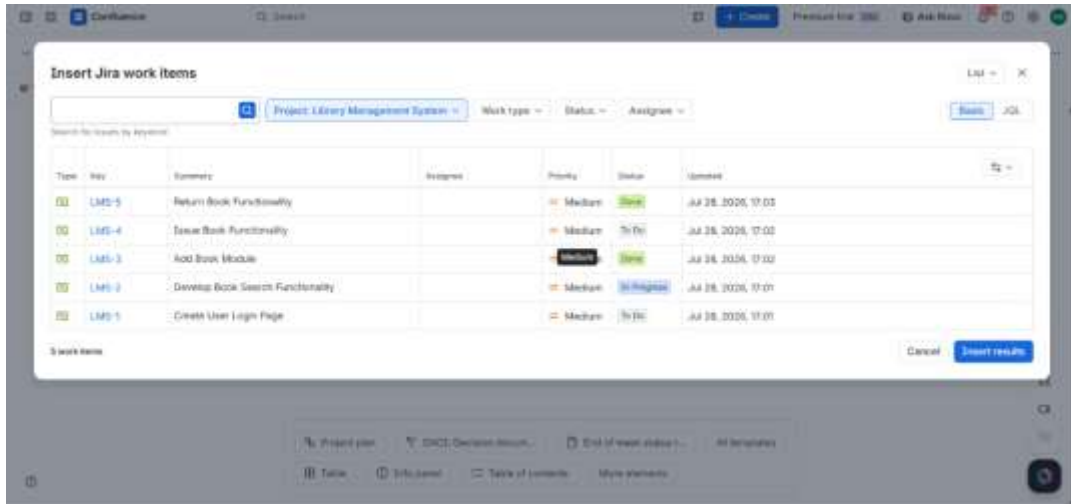
Step 5: Select the Jira Project

1. Select your project
Library Management System
2. Choose
All Work Items
or select the individual issues.
Example issues:

- LMS-1 Create User Login Page
SLOT B-CSA1011-SOFTWARE ENGINEERING

- LMS-2 Develop Book Search
- LMS-3 Issue Book Module
- LMS-4 Return Book Module
- LMS-5 Database Integration

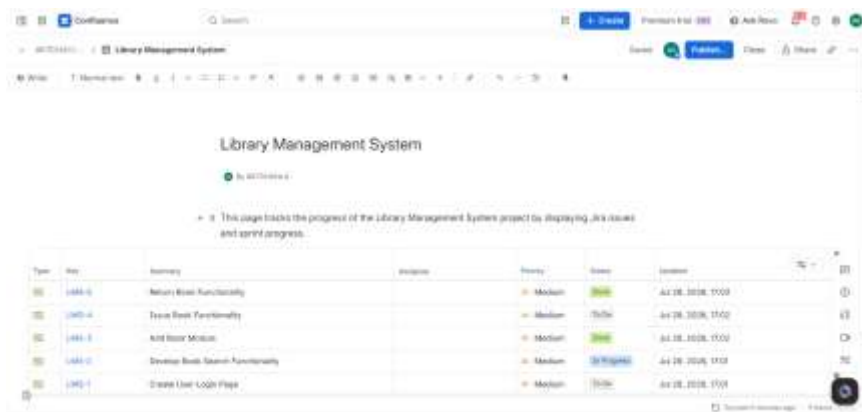
3. Click Insert.



Step 6: Display Embedded Jira Tasks

The selected Jira issues will appear in the Confluence page showing:

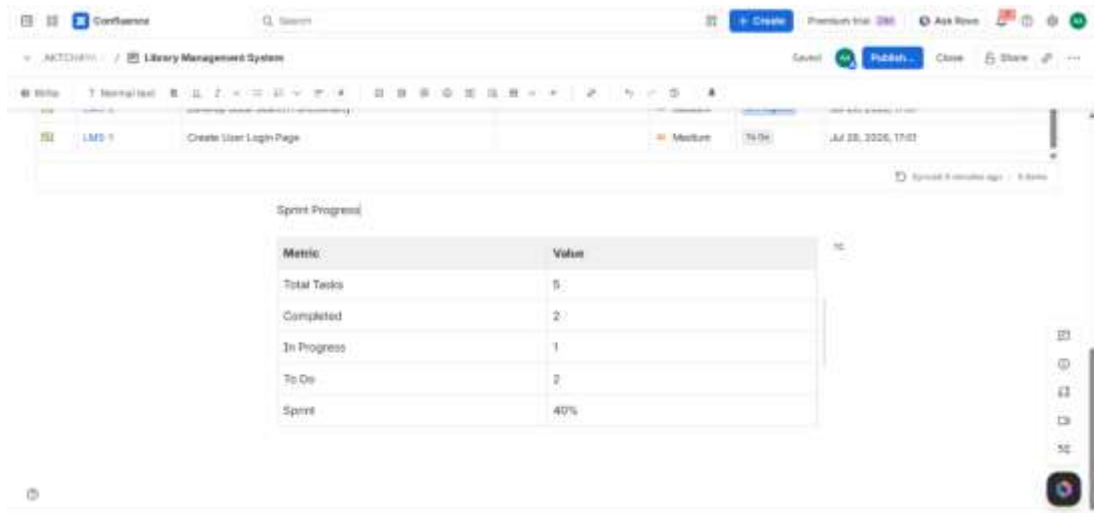
- Issue Key
- Summary
- Status
- Assignee
- Priority



Step 8: Add Progress Bar

Since the Progress Bar macro is unavailable, insert a table below the Jira work items.

Metric	Value
Total Tasks	5
Completed	2
In Progress	1
To Do	2
Sprint Completion	40



Step 9: Publish the Page

1. Click Publish.
2. Verify that the page contains:
 - Page title
 - Introduction
 - Project Details table
 - Embedded Jira work items
 - Status column
 - Sprint Progress table

CEETWYRA A

Published services

140

Store

Library Management System 1

by KETOWYRA A

This page tracks the progress of the Library Management System project by displaying Jira issues and sprint progress.

Field	Details
Project Name	Library Management System
Sprint	Sprint 1
	Software Engineering Lab
	2 Weeks
Manager	KETOWYRA A

Published

Library Management System 1 was successfully published in the KETOWYRA A space

Copy link

EXPERIMENT 12

You are designing a Task Management System for a small team. The system should include the following features:

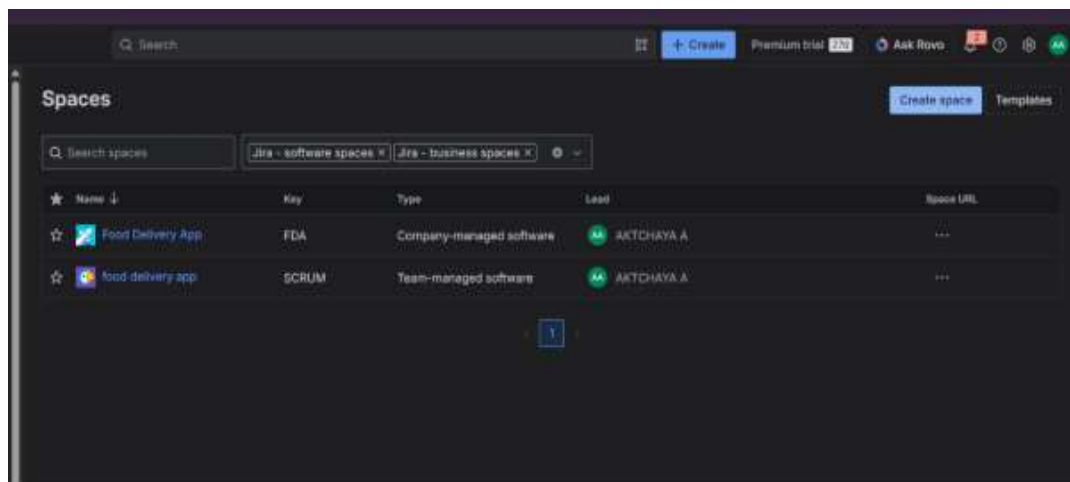
1. User Login and Role Assignment
2. Task Creation and Assignment
3. Task Prioritization and Deadlines
4. Progress Tracking and Reporting

Prioritize these requirements using the MoSCoW and Kano models in Jira.

Steps:

Step 1: Login

- Open a web browser and navigate to the Jira website.
- Login - <https://id.atlassian.com/login>
- Sign in to your Jira account using your registered email address and password.



Step 2: Create a New Project

- Click Create Project.
- Select Scrum
- Enter the project name as Task Management System.
- Choose Team-managed.

- Click Create.

Name your space
Required fields are marked with an asterisk *

Name *
Task Management-1

How your space is managed *
Team-managed

Learn more about space types

Access *
Open

Key *
TM1

Template
Kanban

See details

Step 3: Create an Epic

- Click Create.
- Change Work Type to Epic.
- Enter the Summary:

Task Management System Development

- Click Create.

Create Epic

Space *
Task Management-1 (TM1)

Work type *
Epic

Summary *
Task Management System Development

Cancel Create

Step 4: Create User Story 1

- Click Create
- Select Story
- Summary

User Login and Role Assignment

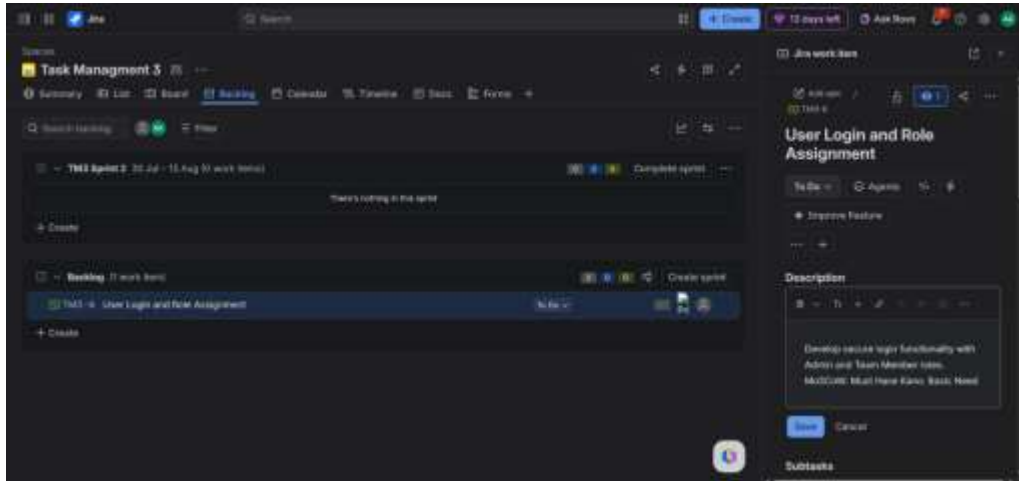
Description

Develop secure login functionality with Admin and Team Member roles.

MoSCoW: Must Have

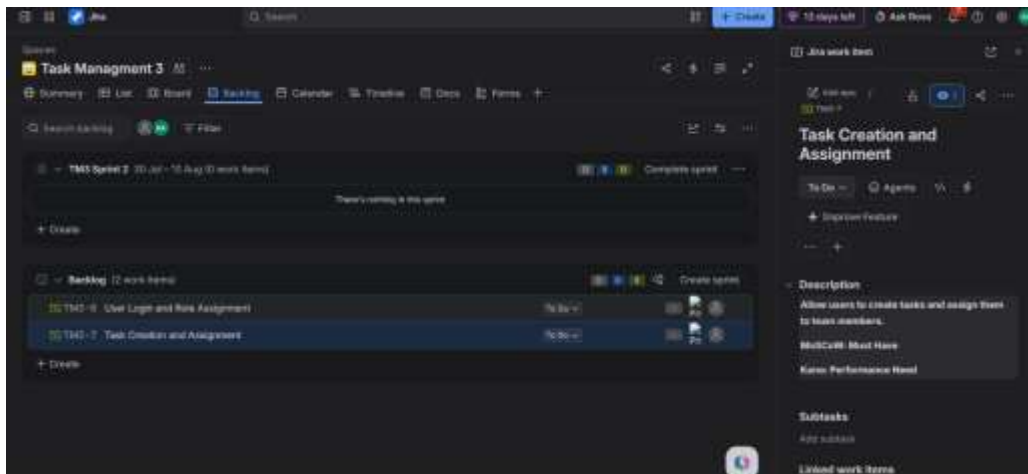
Kano: Basic Need

- Click Create.



Step 5: Create User Story 2

- Summary
Task Creation and Assignment
- Description
Allow users to create tasks and assign them to team members.
MoSCoW: Must Have
Kano: Performance Need
- Click Create.

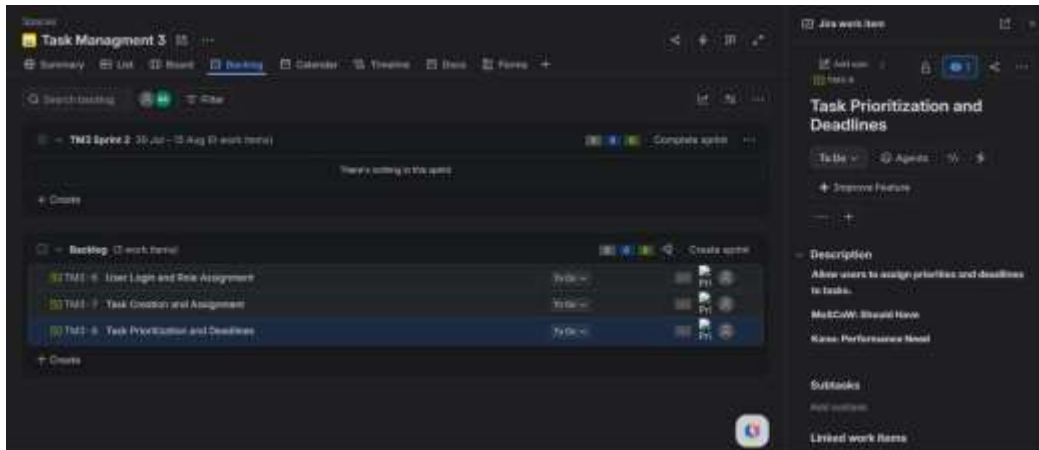


Step 6: Create User Story 3

- Summary
Task Prioritization and Deadlines
- Description
Allow users to assign priorities and deadlines to tasks.
MoSCoW: Should Have

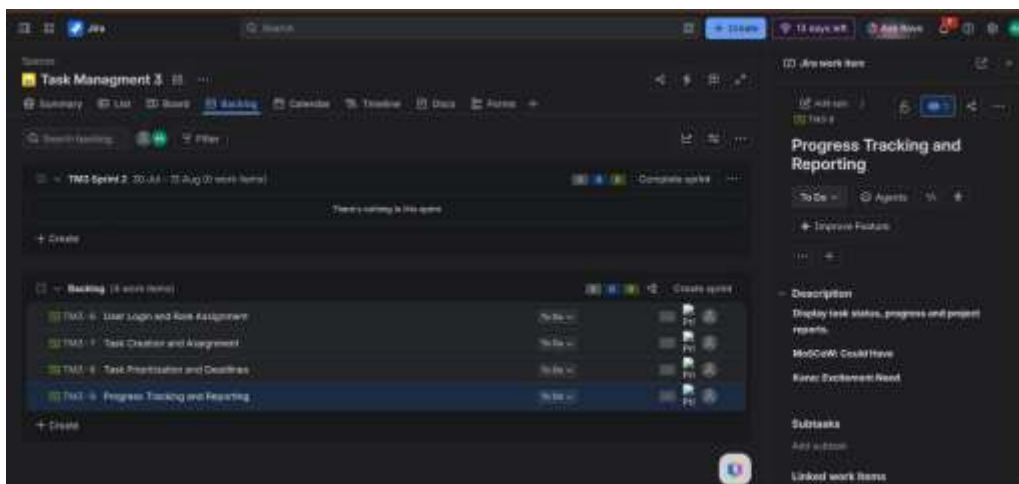
Kano: Performance Need

- Click Create.



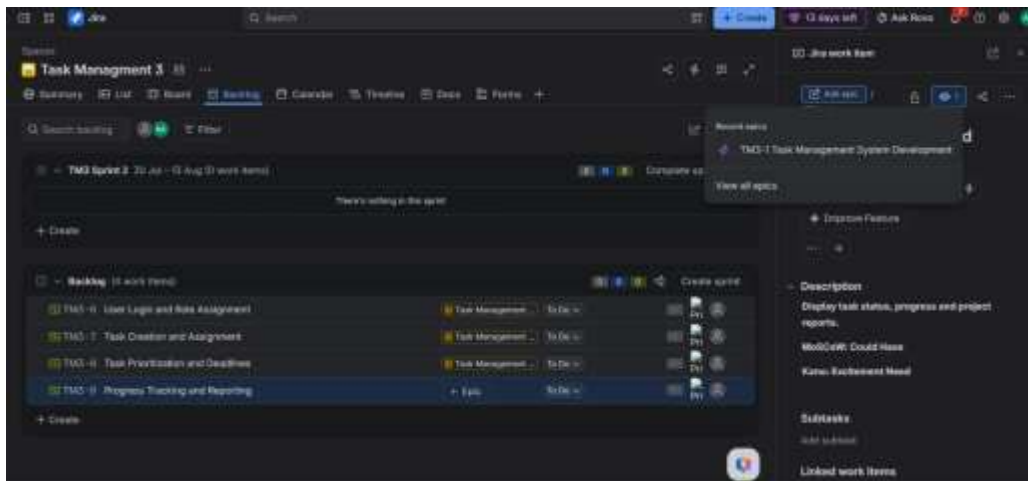
Step 7: Create User Story 4

- Summary
Progress Tracking and Reporting
- Description
Display task status, progress and project reports.
MoSCoW: Could Have
Kano: Excitement Need
- Click Create.



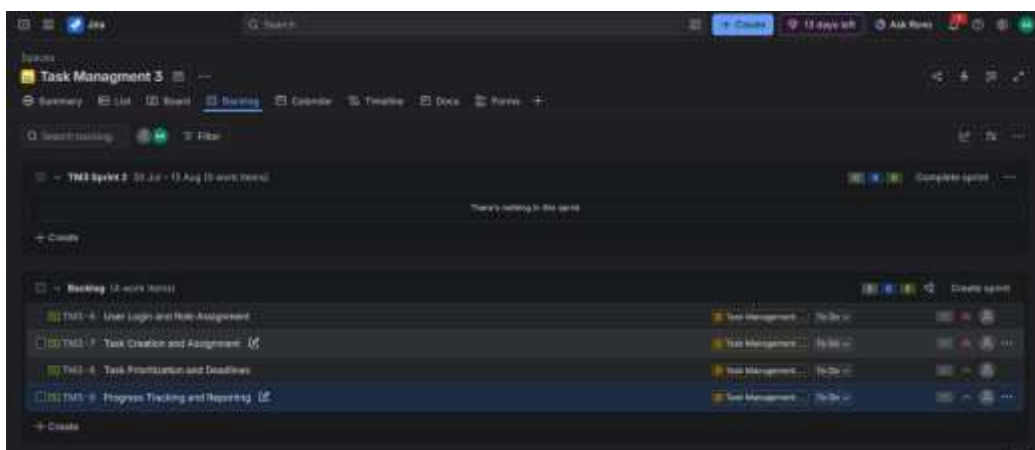
Step 8: Link Stories to Epic

- Open each Story
- Click Add Epic
- Select “Task Management System Development”
- Repeat for all four stories.



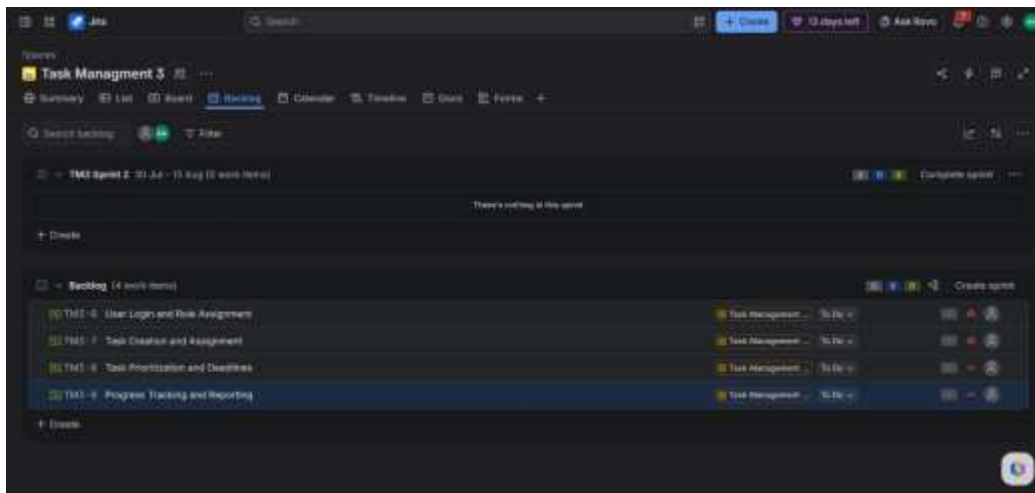
Step 9: Verify the Backlog

- Open Backlog
- You should see
 - User Login and Role Assignment
 - Task Creation and Assignment
 - Task Prioritization and Deadlines
 - Progress Tracking and Reporting



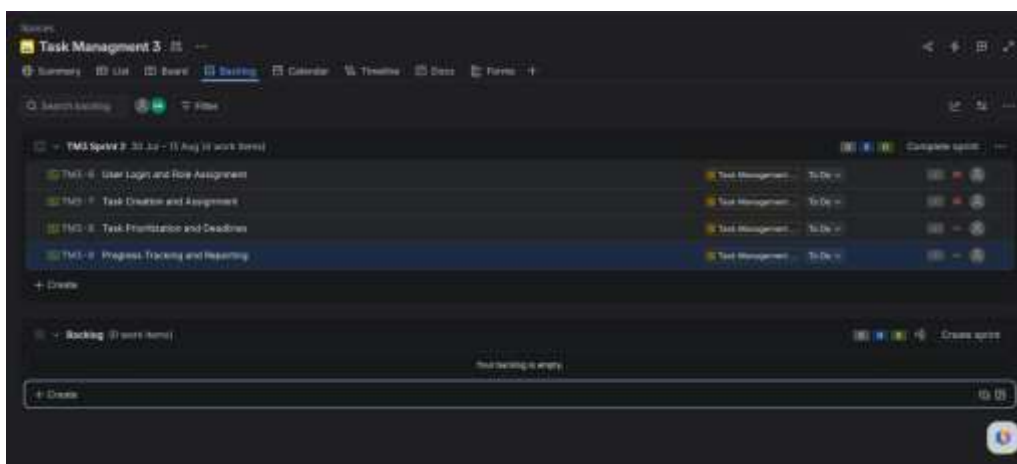
Step 10: Create Sprint

- Click Create Sprint
- A new Sprint is created.



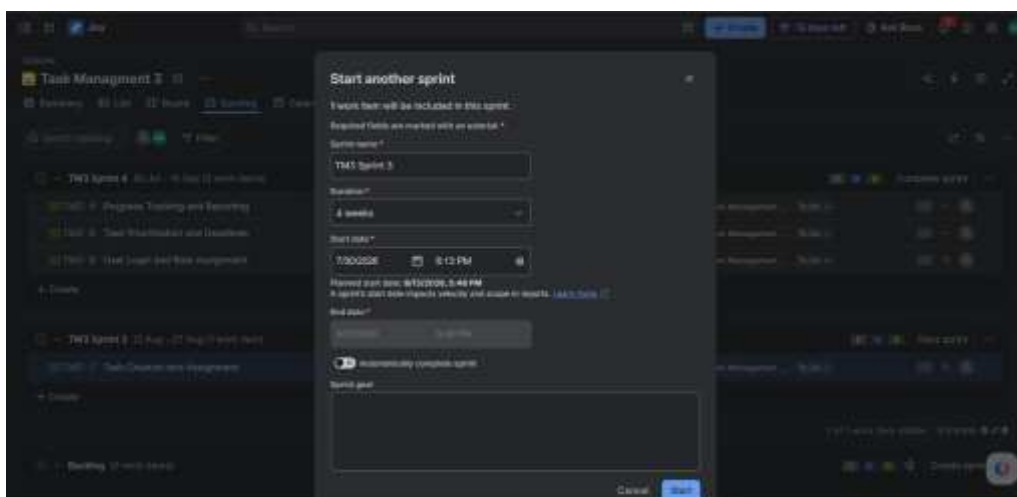
Step 11: Move Stories to Sprint

Drag all four Stories from Backlog to Sprint 2



Step 12: Start Sprint

- Click Start Sprint
- Click Start



Step 13: Prioritize Requirement Using the Moscow Model

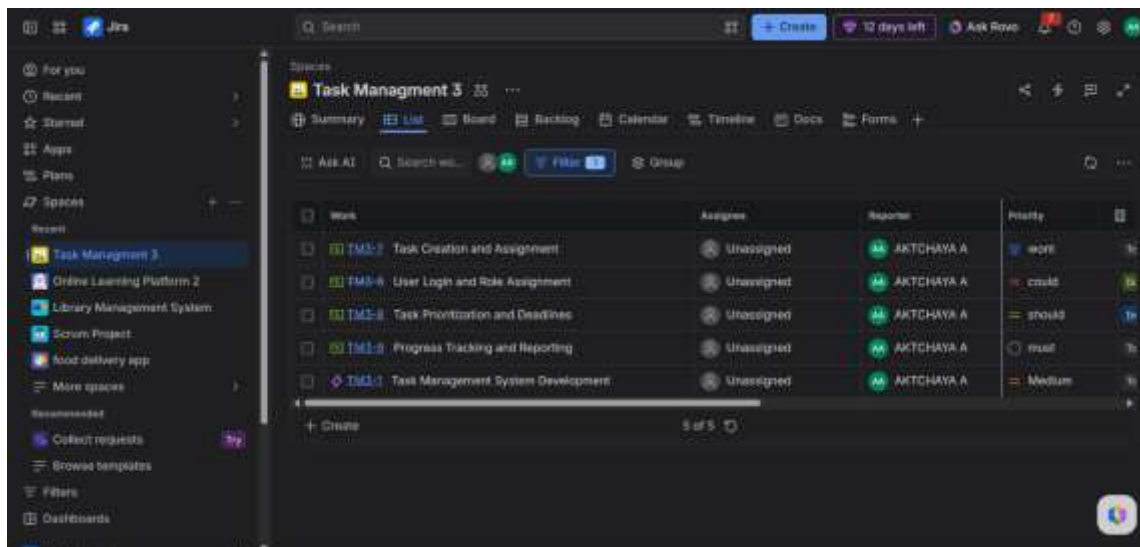
Assign the following priorities to the tasks:

Requirement	MoSCoW Priority
User Login and Role Assignment	Must Have
Task Creation and Assignment	Must Have
Task Prioritization and Deadlines	Should Have
Progress Tracking and Reporting	Could Have



Step 14: Open the Board

Go to Board you will see all four stories in the To Do column.



EXPERIMENT 13

You are tasked with developing an Online Learning Platform. The platform should include the following functionalities:

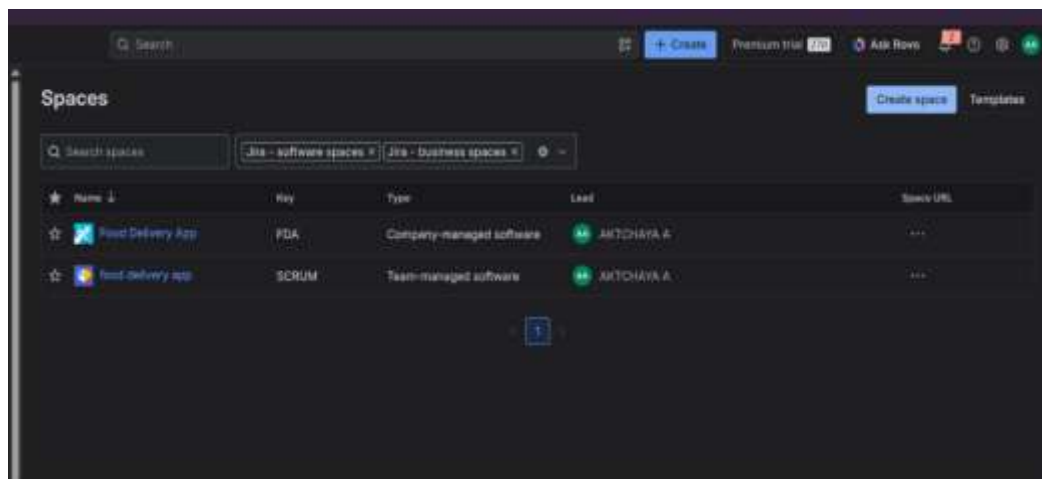
- Course Enrollment and Registration
- Video Lecture Streaming
- Interactive Quizzes and Assignments
- Progress Tracking Dashboard
- Peer-to-Peer Discussion Forums
- Certificate Generation

Use Jira to categorize and prioritize these requirements using MoSCoW and Kano techniques.

Steps:

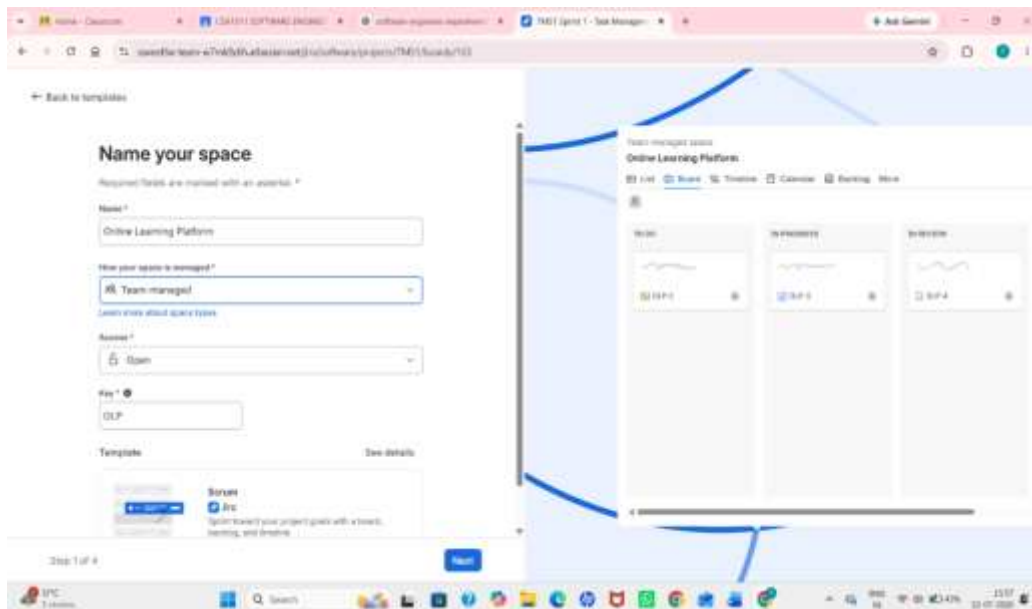
Step 1: Login

- Open a web browser and navigate to the **Jira** website.
- Login - <https://id.atlassian.com/login>
- Sign in to your Jira account using your registered email address and password.



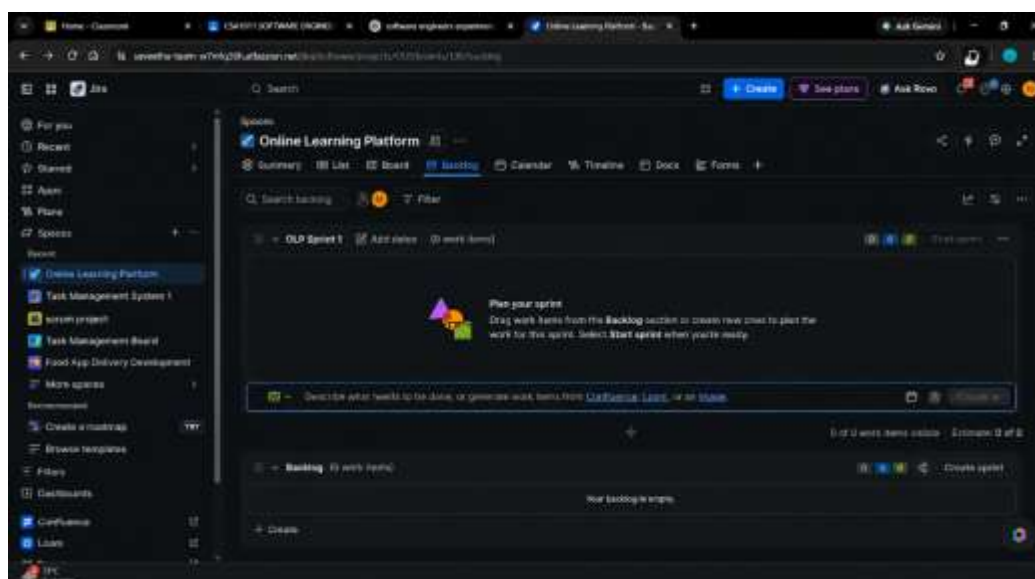
Step 2: Create a New Project

- Click Create Project.
- Select Scrum
- Enter the project name as Task Management System.
- Choose Team-managed.
- Click Create.



Step 3: Open the Backlog

- Select the Online Learning Platform project.
- Click the Backlog tab.
- Create a new sprint if one is not already available.



Step 4: Create User Stories

- Click + **Create** and create the following six stories.
 - **Story 1**

Summary

Course Enrollment and Registration

Description

As a student, I want to register and enroll in courses so that I can access course materials and begin learning.

MoSCoW Priority: Must Have

Kano Category: Basic

Priority: Highest

- **Story 2**

Summary

Video Lecture Streaming

Description

As a student, I want to stream video lectures so that I can learn course content anytime and anywhere.

MoSCoW Priority: Must Have

Kano Category: Basic

Priority: Highest

- **Story 3**

Summary

Interactive Quizzes and Assignments

Description

User Story:

As a student, I want to complete quizzes and assignments so that I can assess my understanding of the course.

MoSCoW Priority: Should Have

Kano Category: Performance

Priority: High

- **Story 4**

Summary

Progress Tracking Dashboard

Description

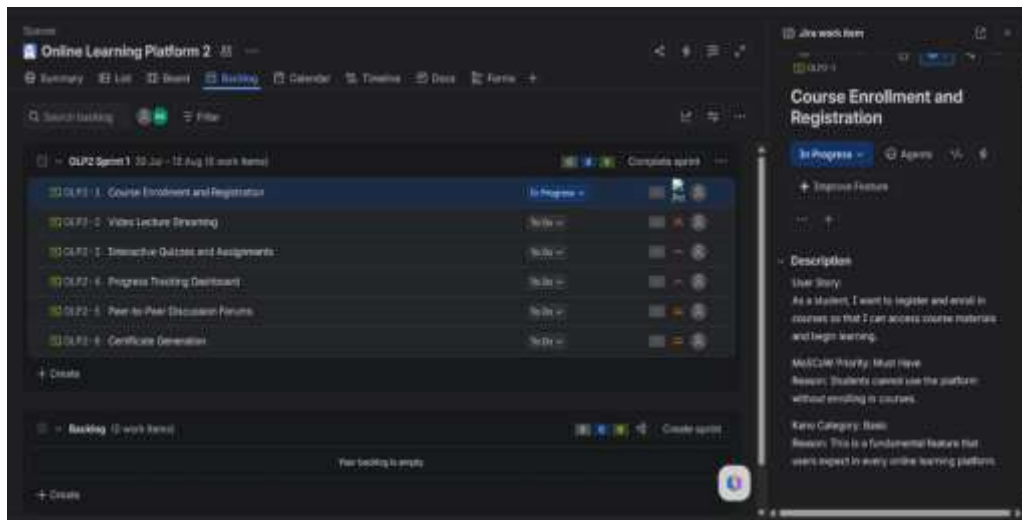
User Story:

As a student, I want to view my learning progress so that I can monitor completed lessons, quiz scores, and overall performance.

MoSCoW Priority: Should Have

Kano Category: Performance

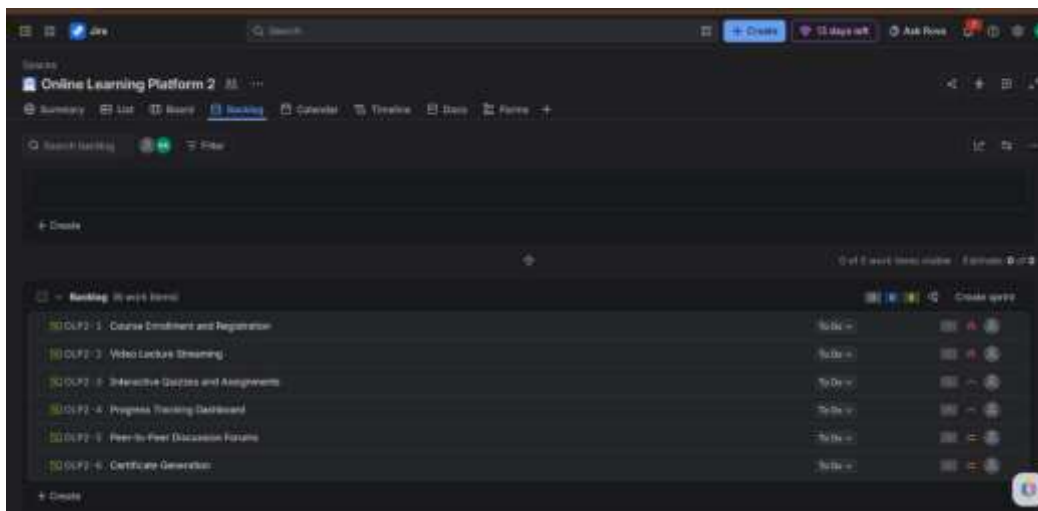
Priority: High



Step 5: Verify All Stories in the Backlog

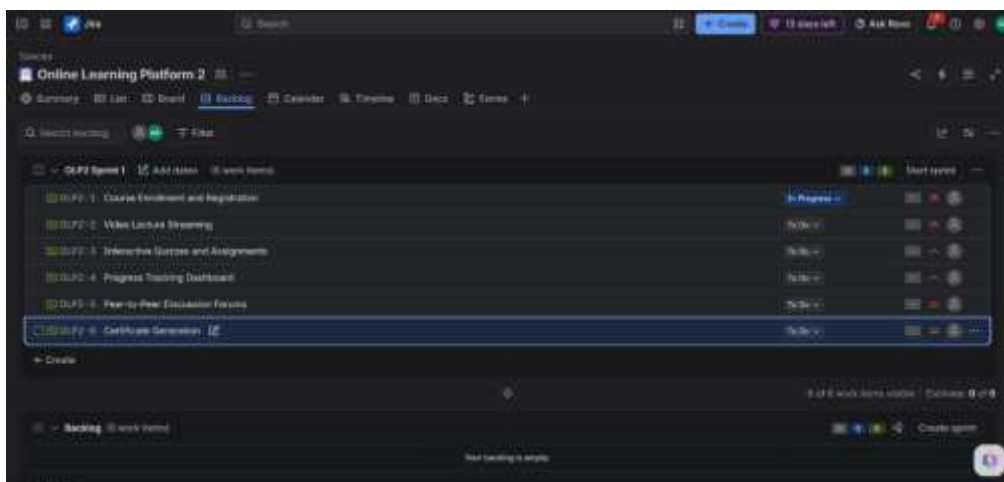
- After creating all six stories:

Ensure all stories are displayed in the Backlog.



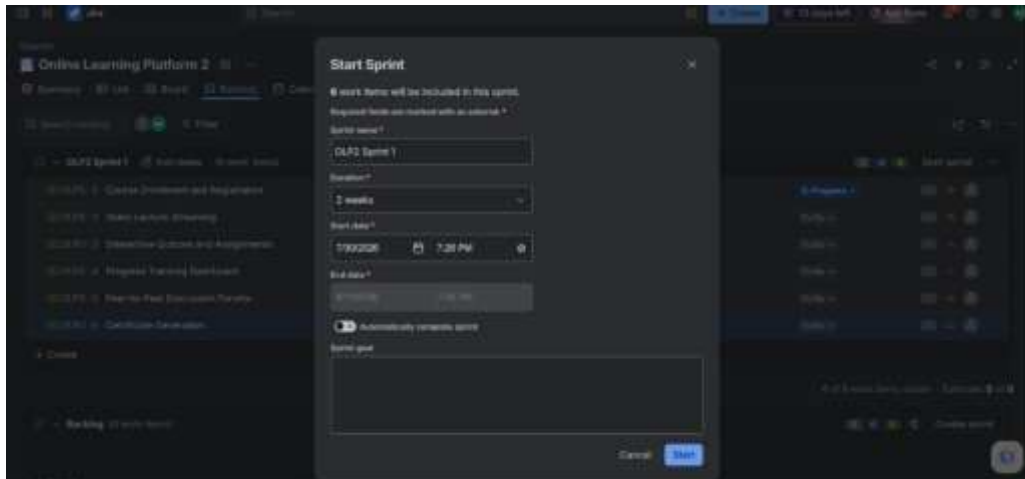
Step 7: Move Stories into Sprint 1

Drag all six stories from the Backlog into **Sprint 1**.



Step 8: Start the Sprint

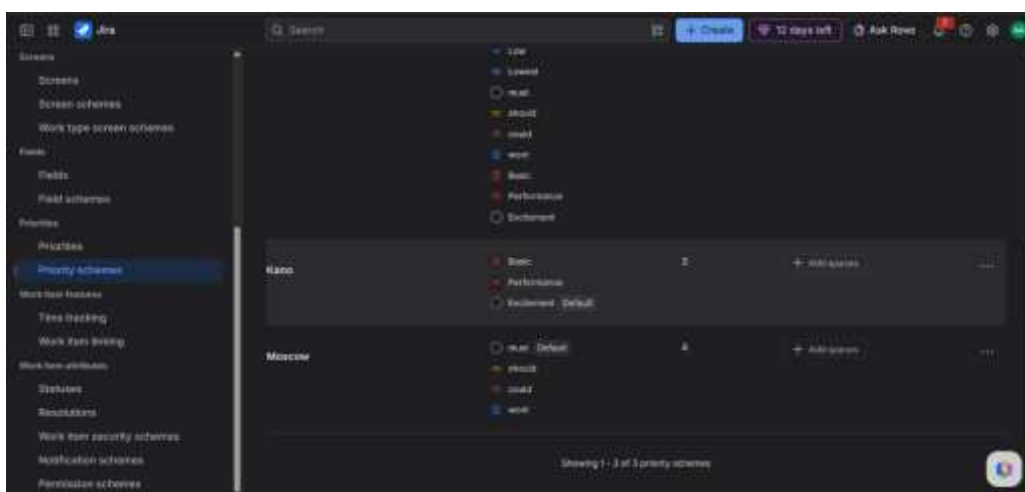
- Click **Start Sprint**.
- Enter:
 - **Sprint Name:** Sprint 1
 - **Duration:** 2 Weeks
 - **Sprint Goal:** Complete the core features of the Online Learning Platform.
- Click **Start**.



Step 9 : Categorize Requirements Using the Kano Model

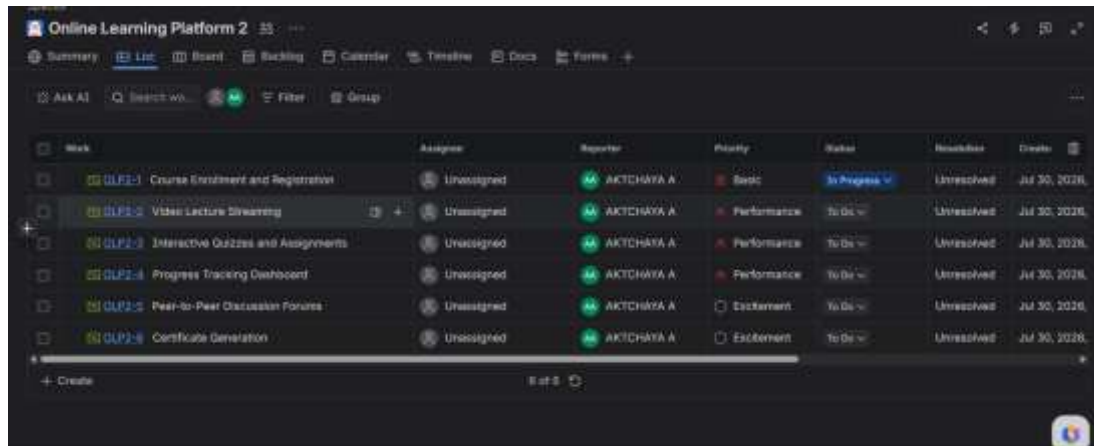
Add another label to each task based on the Kano model.

Requirement	Kano
Course Enrollment and Registration	Basic
Video Lecture Streaming	Performance
Interactive Quizzes and Assignments	Performance
Progress Tracking Dashboard	Performance
Peer-to-Peer Discussion Forums	Excitement
Certificate Generation	Excitement



Step 10: Verify the Active Sprint

- After the sprint starts:
 - Open the **Board**.
 - Verify that Sprint 1 is active.



EXPERIMENT 4

The string is a sequence of characters placed in double quotes (” “). Performing different operations on string data is called String Handling. Strings are immutable. Whenever a change to a String is made, an entirely new String is created. If we want to store a group of characters we can use a char array. String provides various methods to perform different operations on strings. Using Raptor draw a flowchart to display the total number of characters in the string and returns it.

Steps:

Step 1: Open the Raptor Software

Step 2: Create a new flowchart

Step 3: Insert an Input symbol below the Start symbol

Step 4: Double-click the Input symbol below the Input symbol

- Prompt: “Enter a String”
- Variable: str

Step 5: Insert an Assignment symbol below the Input symbol

Step 6: Double-click the Assignment symbol and enter:

- $\text{Count} \leftarrow \text{length_of}(\text{str})$

Step 7: Insert an Output symbol below the Assignment symbol

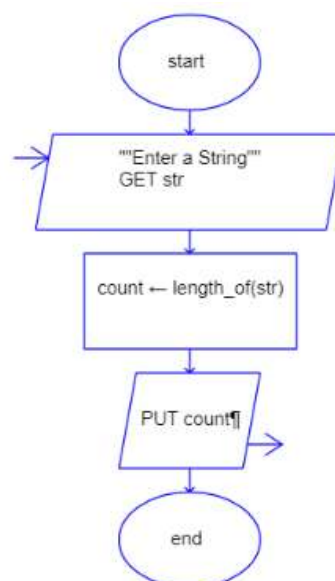
Step 8: Double-click the Output symbol and enter:

- Count

Step 9: Click the (▶) button

Step 10: Enter the input string when prompted (eg., Welcome)

Step 11: Verify the output



EXPERIMENT 5

Using Raptor – Draw and validate a flowchart for a given string of lower case English alphabets. One can choose any two characters in the string and replace all the occurrences of the first character with the second character and replace all the occurrences of the second character with the first character. Using Raptor draw and validate the flowchart lexicographically smallest string that can be obtained by doing this operation at most once.

Steps:

Step 1: Open the **RAPTOR** software.

Step 2: Create a new flowchart.

Step 3: Insert an **Input** symbol below the **Start** symbol. Enter the following:

Prompt

Enter the string, first character and second character

Input Variables

str, ch1, ch2

Step 4: Insert an **Assignment** symbol below the Input symbol.

Enter:

$i \leftarrow 1$

$n \leftarrow \text{Length_Of}(\text{str})$

This initializes the loop counter and stores the length of the string.

Step 5: Insert a **Loop** symbol.

Enter the condition:

$i \leq n$

This loop checks every character in the string.

Step 6: Inside the loop, insert a **Selection** symbol.

Enter the condition:

$\text{str}[i] = \text{ch1}$

- If **True**, insert an Assignment symbol and write:

$\text{str}[i] \leftarrow \text{ch2}$

- If **False**, continue to the next selection.

Step 7: Insert another **Selection** symbol in the False branch.

Enter the condition:

$\text{str}[i] = \text{ch2}$

- If **True**, insert an Assignment symbol and write:

$\text{str}[i] \leftarrow \text{ch1}$

- If **False**, no replacement is performed.

Step 8: Insert an **Assignment** symbol after both branches merge.

Enter:

$i \leftarrow i + 1$

This increments the loop counter.

Connect this Assignment back to the **Loop** symbol.

Step 9: When the loop condition becomes **False**, insert an **Output** symbol.

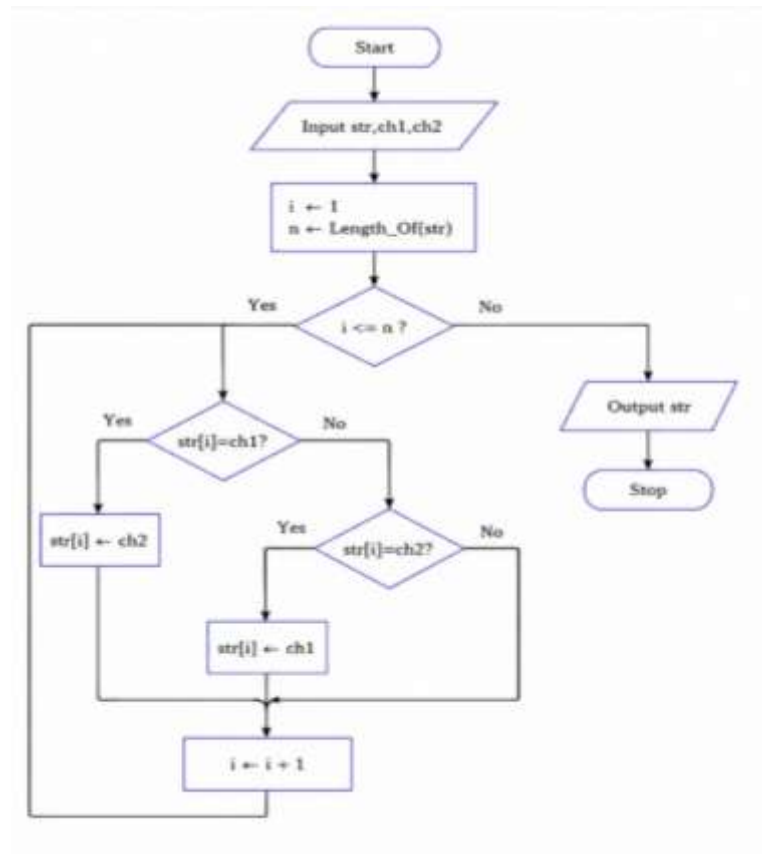
Enter:

Modified String : , str

Step 10: Connect the Output symbol to the **Stop** symbol.

Step 11: Run the flowchart by clicking the **Run (▶)** button.

Step 12: Verify the Output



EXPERIMENT 8

Cyclomatic Complexity in Software Testing is a testing metric used for measuring the complexity of a software program. It is a quantitative measure of independent paths in the source code of a software program. Cyclomatic complexity can be calculated by using control flow graphs or with respect to functions, modules, methods or classes within a software program. Cyclomatic complexity for a flow graph G is $V(G)=E-N+2$. Cyclomatic complexity is a software metric (measurement), used to indicate the complexity of a program, Cyclomatic complexity = $E - N + P$ where, E = number of edges in the flow graph, N = number of nodes in the flow graph, P = number of nodes that have exit points. Find Cyclomatic Complexity for a graph having number of edges as 17, number of nodes as 13 and number of predicate nodes in the flow graph as 5.

Steps:

Step 1: Open the raptor software

Step 2: Create a new flowchart

Step 3: Insert an Input symbol below the start symbol

Prompt:

Enter the number of Edges, Nodes and Predicate Nodes

Variables:

E, N, P

Step 4: Insert an Assignment symbol below the Input symbol

Type:

$CC \leftarrow E - N + P$

where CC represents Cyclomatic Complexity.

Step 5: Insert the Output symbol below the Assignment symbol

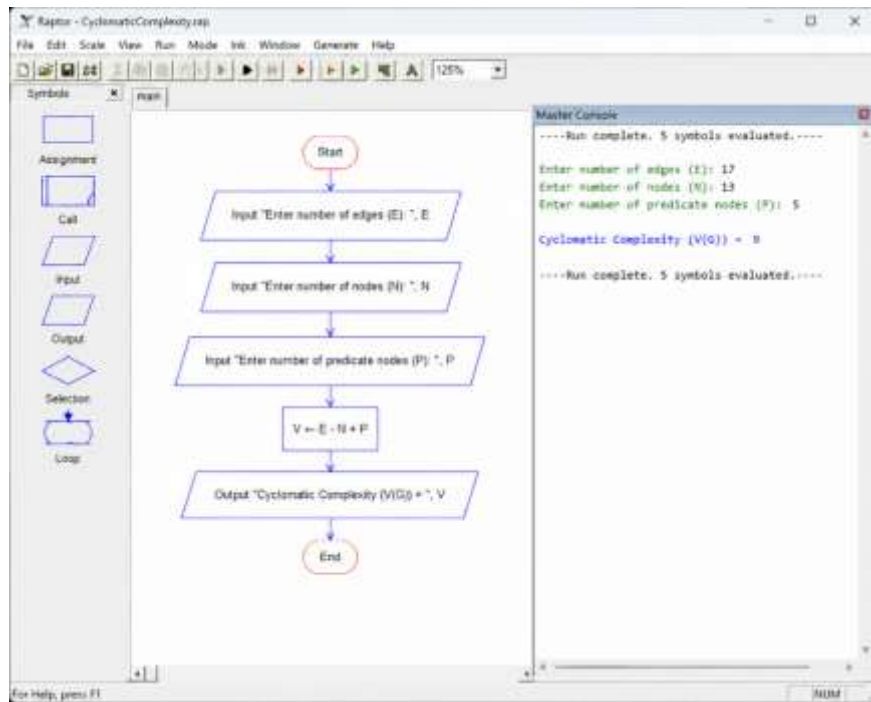
Type:

"Cyclomatic Complexity = ", CC

Step 6: Double-click the Assignment symbol and enter:

$CC \leftarrow E - N + P$

Step 7: Insert an output



EXPERIMENT 6

An online food delivery platform enables customers to browse restaurants, view menus, place food orders, make online payments, and track delivery status. Restaurants use the system to manage their menus and update order status, while delivery agents are responsible for accepting delivery requests and updating delivery progress. Design a Class Diagram, Use case Diagram and Activity Diagram for the Online Food Delivery System showing the relevant classes, their attributes, methods, and relationships.

Steps:

Step 1: Open StarUML

- Launch the **StarUML** application.
- Select **File** → **New** to create a new UML project.
- Save the project with the name **Online Food Delivery System**.

Step 2: Create a Use Case Diagram

- In the **Model Explorer**, right-click **Model**.
- Select **Add Diagram** → **Use Case Diagram**.

Step 3: Add the Actors

Add the following actors:

- Customer
- Restaurant
- Delivery Agent

Step 4: Add the Use Cases

Create the following use cases:

- Register
- Login
- Browse Restaurants
- View Menu
- Add to Cart

- Place Order
- Make Payment
- Track Order
- Manage Menu
- Accept Order
- Update Order Status
- Accept Delivery
- Update Delivery Status

Step 5: Connect the Actors

- Use the **Association** tool to connect each actor to the relevant use cases.
- Save the Use Case Diagram.

Step 6: Create a Class Diagram

- Right-click **Model**.
- Select **Add Diagram** → **Class Diagram**.

Step 7: Add the Classes

Create the following classes:

- Customer
- Restaurant
- Menu
- FoodItem
- Order
- Payment
- Delivery
- DeliveryAgent

Step 8: Add Attributes

Enter suitable attributes for each class.

Examples:

- Customer: customerId, name, phone, address
- Restaurant: restaurantId, restaurantName, location
- FoodItem: itemId, itemName, price
- Order: orderId, orderDate, orderStatus
- Payment: paymentId, amount, paymentStatus
- DeliveryAgent: agentId, name, phone

Step 9: Add Methods

Add operations such as:

- login()
- browseRestaurant()
- placeOrder()
- manageMenu()
- acceptOrder()
- makePayment()
- acceptDelivery()
- updateDeliveryStatus()

Step 10: Create Relationships

- Connect the classes using associations.
- Example:
 - Customer places Order.
 - Restaurant manages Menu.
 - Menu contains FoodItem.
 - Order includes Payment.

- DeliveryAgent delivers the Order.
- Save the Class Diagram.

Step 11: Create an Activity Diagram

- Right-click **Model**.
- Select **Add Diagram** → **Activity Diagram**.

Step 12: Add the Initial Node

- Place the **Initial Node** at the beginning of the diagram.

Step 13: Add Activities

- Create the activities in the following order:
 - Customer Login
 - Browse Restaurants
 - View Menu
 - Select Food
 - Add to Cart
 - Place Order
 - Make Payment
 - Restaurant Accepts Order
 - Food Preparation
 - Delivery Agent Accepts Delivery
 - Deliver Food
 - Customer Receives Order

Step 14: Add Decision Node

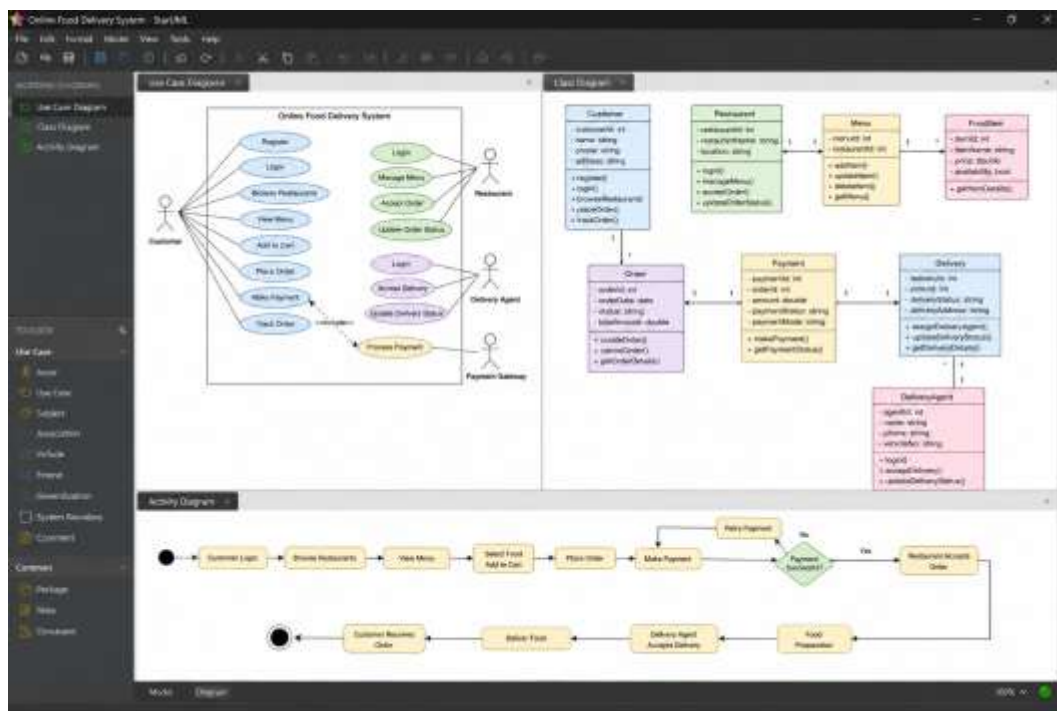
- Insert a decision node after **Make Payment**.
- If the payment is successful, continue to order processing.
- Otherwise, return to the payment step.

Step 15: Connect the Activities

- Use **Control Flow** arrows to connect all activities in sequence.
- Add the **Final Node** to indicate the end of the process.

Step 16: Verify the Diagrams

- Check that all actors, classes, and activities are included.
- Ensure relationships and control flows are correct.
- Confirm that the diagrams follow UML standards.



EXPERIMENT 7

A college employs a large number of security personnel who are assigned duties at various locations across the campus. Each security person follows a duty schedule that includes date, duty start time, duty end time, and assigned location, ensuring that every campus location is covered. If a security person takes leave, it is recorded manually, and at the end of each month, salaries are calculated based on duties performed and leaves taken. Since manual handling of duty allocation, leave management, and salary calculation is time-consuming and complex, the college plans to develop an Online Security Management System to automate these activities. Develop Class Diagram, Use case Diagram and Activity Diagram to represent the workflow of the Online Security Management System.

Steps:

Step 1: Open the UML Tool

- Launch Umbrello UML Modeller (or StarUML).
- Create a New UML Project.
- Save the project as Online Security Management System.

Step 2: Create a Use Case Diagram

- Right-click Model (or use Diagram → New).
- Select Use Case Diagram.

Step 3: Add the Actors

Create the following actors:

- Admin
- Security Personnel
- HR / Accounts

Step 4: Add the Use Cases

Add the following use cases:

- Login
- Manage Security Personnel
- Manage Locations
- Assign Duty
- View Duty Schedule
- Apply for Leave

- Approve/Reject Leave
- View Leave Status
- Generate Monthly Salary
- View Reports
- Logout

Step 5: Connect Actors and Use Cases

- Use the Association connector.
- Connect each actor to the corresponding use cases.
- Save the Use Case Diagram.

Step 6: Create a Class Diagram

Create a new Class Diagram.

Step 7: Add the Classes

Create the following classes:

- SecurityPersonnel
- DutySchedule
- Location
- Leave
- Salary
- Payment

Step 8: Add Attributes

- **Enter suitable attributes.**
- **Example:**
 - SecurityPersonnel: personID, name, phone, address
 - DutySchedule: dutyID, date, startTime, endTime
 - Location: locationID, locationName
 - Leave: leaveID, leaveDate, leaveType, status
 - Salary: salaryID, month, basicSalary, deduction, netSalary
 - Payment: paymentID, amount, paymentDate, status

Step 9: Add Methods

- Add operations such as:
 - addPersonnel()
 - assignDuty()
 - updateDuty()
 - applyLeave()
 - approveLeave()
 - calculateSalary()
 - generateSalary()
 - makePayment()

Step 10: Create Relationships

- Connect the classes using associations.
- Example:
 - SecurityPersonnel → DutySchedule
 - DutySchedule → Location
 - SecurityPersonnel → Leave
 - SecurityPersonnel → Salary
 - Salary → Payment
 - Save the Class Diagram.

Step 11: Create an Activity Diagram

- Create a new Activity Diagram.

Step 12: Add the Initial Node

- Place the Initial Node at the beginning.

Step 13: Add Activities

- Create the activities in the following order:
 - Admin Login
 - Manage Security Personnel
 - Manage Locations
 - Assign Duty Schedule
 - Security Personnel Views Duty Schedule
 - Apply for Leave
 - Admin Approves/Rejects Leave

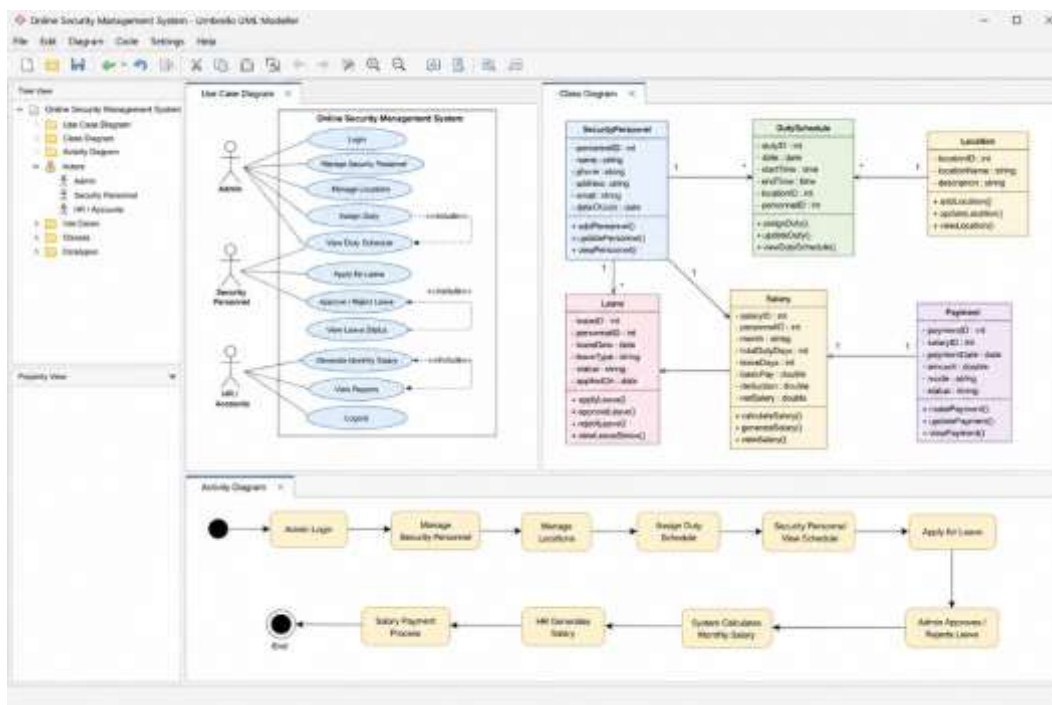
- System Calculates Monthly Salary
- HR Generates Salary
- Salary Payment Process

Step 14: Add Decision Node

- Insert a decision after Apply for Leave:
 - If approved, continue to salary calculation.
 - If rejected, update leave status and continue.

Step 15: Connect the Activities

- Use Control Flow arrows to connect each activity.
- Add the Final Node at the end.



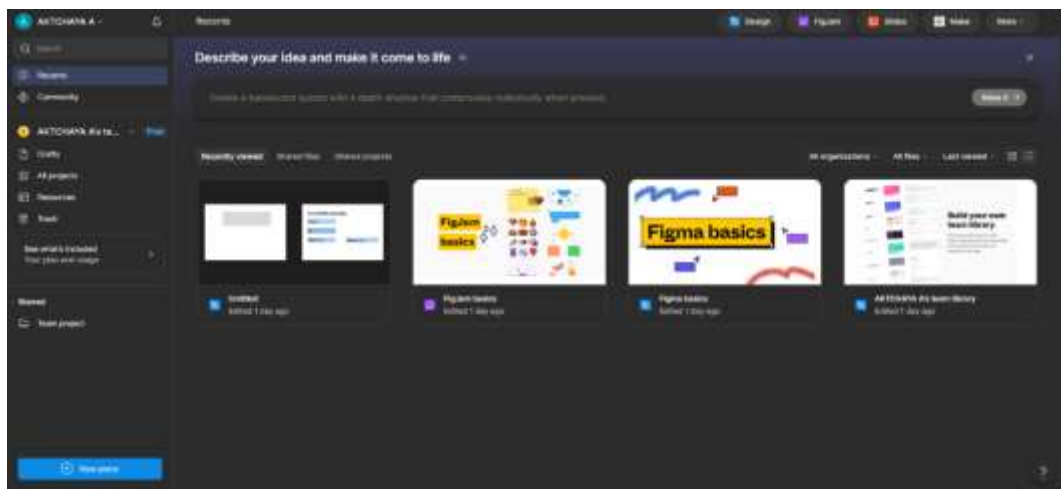
EXPERIMENT 3

The stakeholders have conflicting views on the user interface design for an E-Commerce mobile app. Create a prototype using Figma tool to discuss with the stakeholders to get their feedback and approval.

Steps:

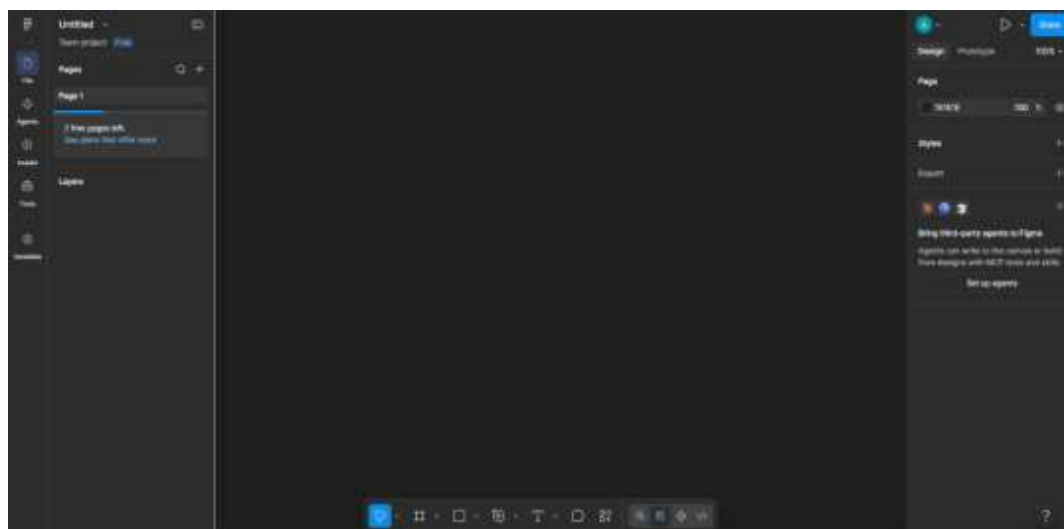
Step 1: Open Figma

- Open your web browser.
- Go to <https://www.figma.com>.
- Sign in using your Figma account.



Step 2: Create a New Design

- Click New Design File.
- A blank design canvas will open.



Step 3: Create Home Screen

- Use Text Tool (T) and type: Shop Easy
- Add:
 - Search Bar
 - Cart Icon
 - Profile Icon
- Add Banner
- Text: Big Sale - Up to 50% Off
- Add Categories
- Create 4 small boxes:
 - Electronics
 - Fashion
 - Shoes
 - Books
- Add Product Cards: Create 2 product cards.
 - Product 1
 - Wireless Headphones
 - ₹1,999
 - Add to Cart
 - Product 2
 - Running Shoes
 - ₹2,499
 - Add t



Step 4: Create Product Details Screen

1. Press F.
2. Create another mobile frame.
3. Rename it Product Details.
 - Add Product Image: Draw a large rectangle.
 - Add Details:
 - Wireless Headphones
 - Price: ₹1,999
 - Rating: ★★★★★☆
 - Description:
High-quality wireless headphones.
 - Add Button: Buy Now



Step 5: Create Cart Screen

- Create a new frame.
- Title: Shopping Cart
- Add:
 - Wireless Headphones
 - ₹1,999

- Quantity: 1
- Button: Checkout



Step 6: Create Checkout Screen

- Create another frame.
- Fields:
 - Name
 - Address
 - Phone Number
 - Payment Options:
 - UPI
 - Card
 - Cash on Delivery
- Button: Place Order



Step 7: Create Order Confirmation Screen

- Create another frame.
- Add:
 - ✓ Order Placed Successfully
 - Order ID: ORD12345
 - Estimated Delivery:3 Days
- Button: Continue Shopping



EXPERIMENT 10:

Use the following requirements for a Library Management System:

- **Add a feature to search books by title and author.**
- **Implement an online book reservation system.**
- **Generate monthly reports on borrowed books for administrators.**
- **Enable email notifications for overdue books.**
- **Add support for QR code scanning for borrowing and returning books.**
- **Create a user-friendly dashboard for librarians.**
- **Allow users to review and rate books.**
- **Integrate a chatbot for user assistance.**
- **Develop a mobile app version of the system.**
- **Provide multi-language support.**

Categorize each requirement using MOSCOW Method (Must-Have, Should-Have, Could-Have, or Won't-Have) based on the following criteria:

- **Impact on the users and stakeholders.**
- **Feasibility considering time, budget, and resource constraints.**

Finally Submit the completed Google Sheet or Excel file with all requirements categorized and justified.

Steps:

Step 1: Read the Problem Statement

- Read the given requirements of the Library Management System carefully.
- Understand the purpose and functionality of each requirement.
- Identify the features that need to be categorized using the MoSCoW method.

Step 2: Open Google Sheets or Microsoft Excel

- Launch Google Sheets or Microsoft Excel.
- Create a new spreadsheet.
- Save the file with the name "Library Management System – MoSCoW Prioritization."

Step 3: Create the Spreadsheet Structure

- Create the required columns in the spreadsheet.
- Include the following column headings:
 - Requirement ID
 - Requirement Description
 - MoSCoW Category
 - Justification

Step 4: List the Requirements

- Enter all the given requirements into the Requirement Description column.
- Assign a unique Requirement ID for each requirement.
- Verify that all requirements have been entered correctly.

Step 5: Categorize the Requirements

- Analyze each requirement based on its importance to the system.
- Assign one of the following MoSCoW categories:
 - Must-Have
 - Should-Have
 - Could-Have
 - Won't-Have
- Ensure that each requirement is placed in only one category.

Step 6: Justify Each Category

- Provide a brief justification for every categorized requirement.
- Consider the following factors while writing the justification:
 - Impact on users and stakeholders.
 - Feasibility based on available time, budget, and resources.
- Ensure that the justification clearly explains the assigned priority.

Step 7: Review the Categorization

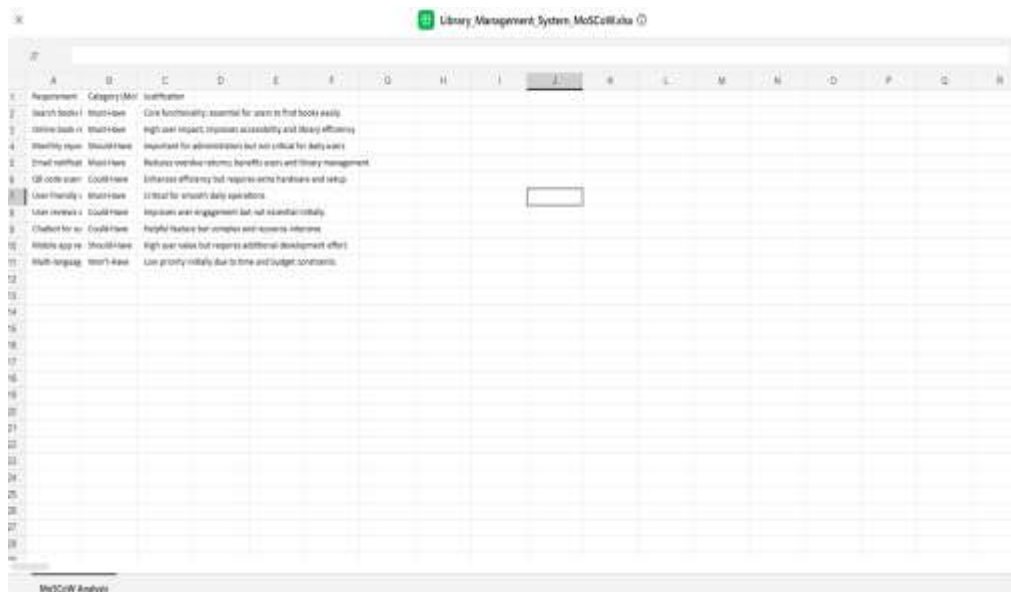
- Verify that every requirement has a MoSCoW category.
- Check that all justifications are meaningful and appropriate.
- Ensure there are no missing or duplicate entries.

Step 8: Format the Spreadsheet

- Apply suitable formatting to improve readability.
- Highlight the column headings.
- Adjust column widths and align the content properly.
- Ensure the spreadsheet is neat and well-organized.

Step 9: Save the Spreadsheet

- Save the completed spreadsheet.
- Verify that all data has been stored correctly.



The screenshot shows a spreadsheet titled "Library Management System, MoSCoW Analysis". The table contains the following data:

Requirement	Category (MoSCoW)	Justification
Search books	Must-Have	Core functionality; essential for users to find books easily.
Online book res	Must-Have	High user impact; improves accessibility and library efficiency.
Staffing system	Should-Have	Important for administrative tasks but not critical for daily work.
Email notifications	Should-Have	Reduces administrative overheads and improves library management.
QR code system	Could-Have	Enhances efficiency but requires extra hardware and setup.
User feedback	Should-Have	Crucial for system's daily operations.
User reviews	Could-Have	Improves user engagement but not immediately critical.
Chatbot for support	Could-Have	Helpful feature for complex and recurring inquiries.
Mobile app version	Should-Have	High user value but requires additional development effort.
Multi-language	Won't Have	Low priority initially due to time and budget constraints.